SCL Compiler + SCL Studio — คู่มือสำหรับพัฒนาต่อ

สรุปโปรแกรมอย่างละเอียด · สถาปัตยกรรม · Virtual PLC และการ Download · UI · แหล่งข้อมูล · คำแนะนำ

18 ตุลาคม 2026

สารบัญ

0. สารบัญและวิธีใช้เอกสาร (00_index)
1. ภาพรวมโครงการ (01_overview)
2. สถาปัตยกรรม Compiler (02_architecture)
3. ภาษา SCL ที่รองรับ (03_language)
4. รายละเอียดภายใน Compiler (04_compiler_internals)
5. รูปแบบไฟล์ .svpl และ Memory model (05_image_format)
6. Instruction set และ Call convention (06_instruction_set)
7. โครงสร้าง Virtual PLC เพื่อรองรับการ Download (07_virtual_plc)
8. User Interface (08_user_interface)
9. การทดสอบ (09_testing)
10. คู่มือพัฒนาต่อ (10_developer_guide)
11. คำแนะนำและ Roadmap (11_recommendations)
12. แหล่งข้อมูลที่ใช้สร้าง (12_references)
13. ภาคผนวก (13_appendix)

สารบัญและวิธีใช้เอกสาร

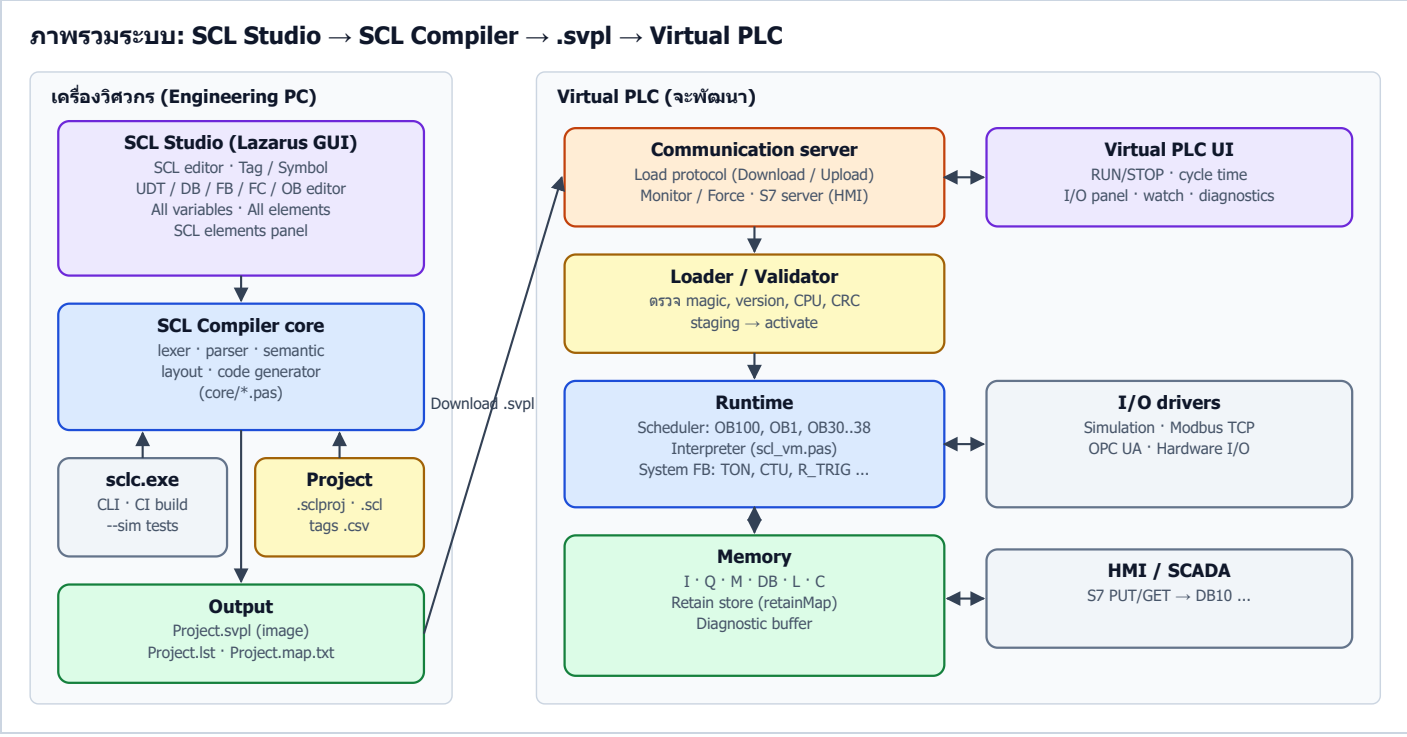
เอกสารชุดนี้สรุปโปรแกรม SCL Compiler + SCL Studio ที่สร้างด้วย Lazarus / Free Pascal อย่างละเอียด เพื่อใช้พัฒนาต่อ รวมถึงการออกแบบ Virtual PLC ที่รองรับการ Download ไฟล์ image จาก IDE

ไฟล์	หัวข้อ	เหมาะกับ
01_overview	ภาพรวมโครงการ สถานะ สถิติ โครงสร้างโฟลเดอร์ เริ่มใช้งาน	ทุกคน
02_architecture	สถาปัตยกรรม compiler, pipeline, unit และ dependency	ผู้พัฒนา compiler
03_language	ภาษา SCL ที่รองรับ กฎตาม CPU และข้อจำกัด	ผู้ใช้ / ผู้พัฒนา
04_compiler_internals	รายละเอียดภายใน: lexer, parser, type system, layout, codegen	ผู้พัฒนา compiler
05_image_format	รูปแบบไฟล์ .svpl และ memory model	ผู้พัฒนา Virtual PLC
06_instruction_set	Instruction set, call convention, system FB	ผู้พัฒนา Virtual PLC
07_virtual_plc	โครงสร้าง Virtual PLC และการรองรับ Download (protocol, validation, mode)	ผู้พัฒนา Virtual PLC
08_user_interface	UI ปัจจุบันของ IDE และ UI พื้นฐานที่ต้องมี (IDE online + Virtual PLC)	ผู้ออกแบบ UI
09_testing	ชุดทดสอบ ภาษา script และวิธีเพิ่ม test	ผู้พัฒนา / QA
10_developer_guide	คู่มือแก้ไข/เพิ่ม feature, coding convention, ข้อควรระวัง FPC/LCL	ผู้พัฒนา
11_recommendations	คำแนะนำ ความเสี่ยง และ Roadmap	หัวหน้าโครงการ
12_references	แหล่งข้อมูลที่ใช้สร้างแต่ละหัวข้อ และเอกสารที่ควรใช้ตรวจสอบ	ทุกคน
13_appendix	Glossary, รูปแบบไฟล์ project/tag, คำสั่ง CLI, คีย์ลัด	ทุกคน

ทุกหัวข้อมีทั้งไฟล์ .txt (อ่านด้วย text editor) และ .pdf (มีรูปภาพ) และมีไฟล์รวม SCL_Compiler_Handbook.pdf / .txt รูปภาพทั้งหมดอยู่ในโฟลเดอร์ images/ ทั้งแบบ .svg (แก้ไขได้) และ .png

> สถานะความถูกต้อง: compiler ถูกทดสอบด้วยโปรแกรมจริง (FluffSilo, 14 block) และชุดทดสอบภาษา บน reference Virtual PLC ที่อยู่ในโปรเจกต์ แต่ยังไม่ได้เทียบผลกับ TIA Portal หรือ PLC จริง ส่วนการออกแบบ Virtual PLC / Download ในหัวข้อ 07 เป็นข้อเสนอ (ยังไม่ได้ implement)

ภาพรวมโครงการ



รูป 1.1 ภาพรวม: SCL Studio → SCL Compiler → .svpl → Virtual PLC

เป้าหมาย

- แปลงโปรแกรม SCL ของ Siemens (S7-300 / S7-400 / S7-1200 / S7-1500) ในรูปแบบ "External source" ของ TIA Portal ให้เป็น bytecode
- ให้มี IDE ภาษาไทย/อังกฤษสำหรับเขียน แก้ไข ตรวจ และ build โปรแกรม SCL โดยไม่ต้องใช้ TIA Portal
- ผลลัพธ์เป็นไฟล์ image (.svpl) ที่ Virtual PLC (พัฒนาภายหลัง) โหลดและรันได้
- โครงสร้างหน่วยความจำ DB แบบ standard access ตรงกับ S7 เพื่อให้ HMI เดิมอ่าน/เขียนด้วย address เดิมได้

สิ่งที่ทำเสร็จแล้ว

ส่วน	สถานะ	หมายเหตุ
Lexer / Parser SCL ครบทุก block	เสร็จ	TYPE, DATA_BLOCK, FUNCTION, FUNCTION_BLOCK, ORGANIZATION_BLOCK
Semantic + type checking ตาม CPU	เสร็จ	error พร้อม file(line,col), ไม่หยุดที่ error แรก
Memory layout แบบ S7 standard	เสร็จ	DB_HMI 82 byte ตรงกับ HMI map ทุก offset
Code generator → bytecode	เสร็จ	91 opcode, system FB 15 ชนิด
ไฟล์ image .svpl + listing + memory map	เสร็จ	reader/writer แยก unit ใช้กับ Virtual PLC ได้
Reference Virtual PLC (interpreter)	เสร็จ (ทดสอบ)	ใช้ทดสอบ compiler; เป็นต้นแบบของ Virtual PLC จริง
CLI sclc	เสร็จ	compile, import, dump, sim/test, find-decl
IDE SCL Studio	เสร็จ	editor, tag, interface grid, views, find declaration, highlight, elements panel, recent files
Virtual PLC + Download	ยังไม่ทำ	ออกแบบไว้ในหัวข้อ 07

สถิติ

รายการ	ค่า
ภาษา / เครื่องมือ	Free Pascal 3.2.2, Lazarus 4.x (LCL, SynEdit)
จำนวน unit	17 ไฟล์ (core 12, GUI 4, CLI 1) ประมาณ 15,400 บรรทัด
ไฟล์ executable	bin/sclc.exe (~1.9 MB), bin/SCLStudio.exe (~42 MB รวม debug info)
ชุดทดสอบ	FluffSilo 65 checks, FeatureTest 44 checks, ErrorTest 10 errors — ผ่านทั้งหมด

โครงสร้างโฟลเดอร์

```
SCLCompiler\
  build.bat            build ทั้ง CLI และ GUI
  run_tests.bat        compile ตัวอย่าง + รันชุดทดสอบ
  README.md            คู่มือย่อ
  bin\                 sclc.exe, SCLStudio.exe
  core\                compiler + image format + reference VM (ไม่มี GUI)
  cli\sclc.lpr         command line
  gui\                 SCLStudio.lpi + sclide_*.pas
  examples\FluffSilo   โปรเจกต์จริง S7-1200 + tests\sequence.txt
  examples\FeatureTest ทดสอบ feature ภาษา (S7-1500)
  examples\ErrorTest   ทดสอบข้อความ error
  docs\SVPL_Spec.md    สเปก image (ฉบับย่อ)
  docs\handbook\       เอกสารชุดนี้ (.txt .pdf images\)
```

เริ่มใช้งานเร็ว

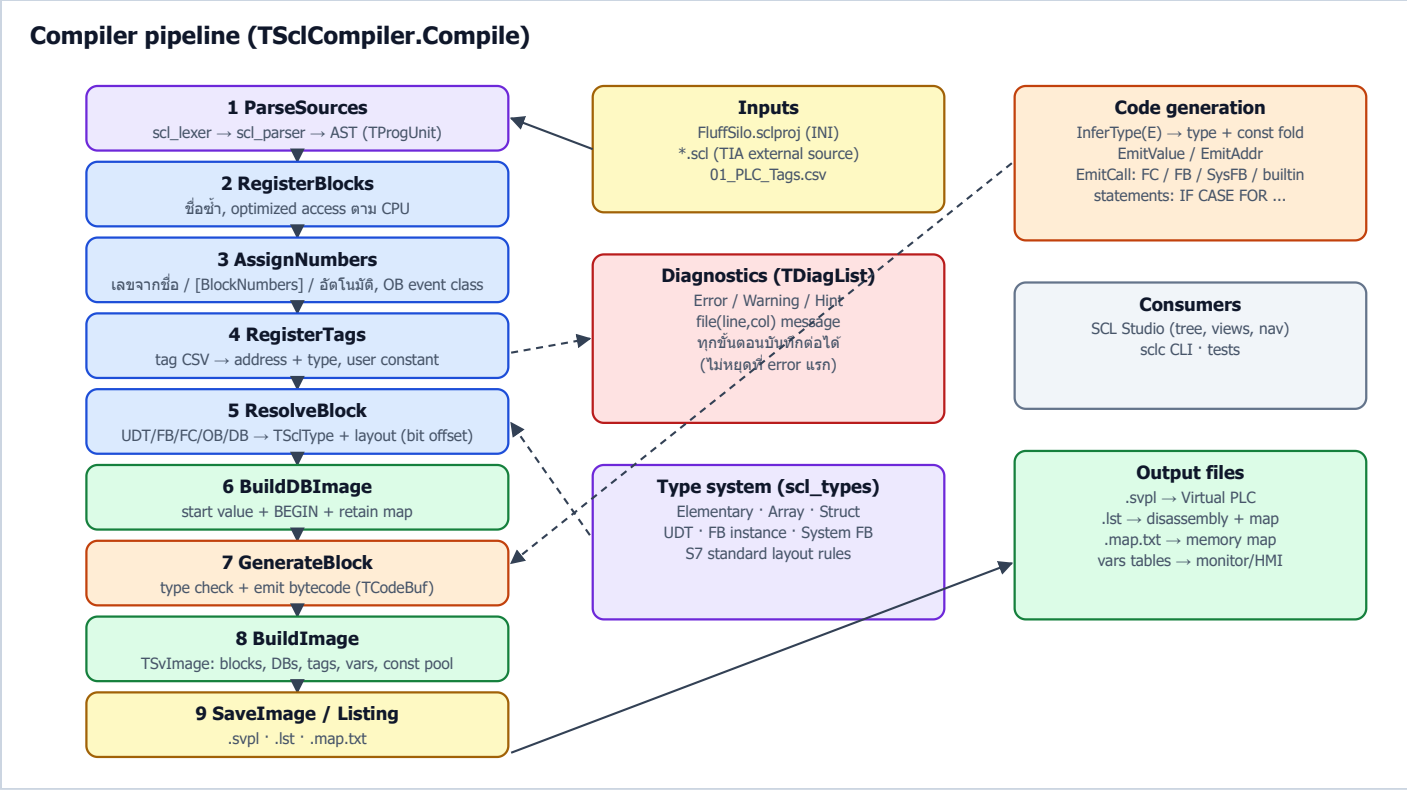
1. build: รัน build.bat (ต้องมี Lazarus ที่ C:\lazarus)
2. เปิด bin\SCLStudio.exe → File > Open project → examples\FluffSilo\FluffSilo.sclproj
3. กด Compile (F7) ดูผลใน Messages, Build (F9) ได้ output\FluffSilo.svpl
4. ทดสอบบน reference VM: sclc --sim output\FluffSilo.svpl --script tests\sequence.txt

---

แหล่งข้อมูลของหัวข้อนี้

- โค้ดในโฟลเดอร์ SCLCompiler (สถิติคำนวณจากไฟล์จริง)
- ผลการรัน run_tests.bat

สถาปัตยกรรม Compiler



รูป 2.1 ขั้นตอนการ compile (TSclCompiler.Compile)

หลักการออกแบบ

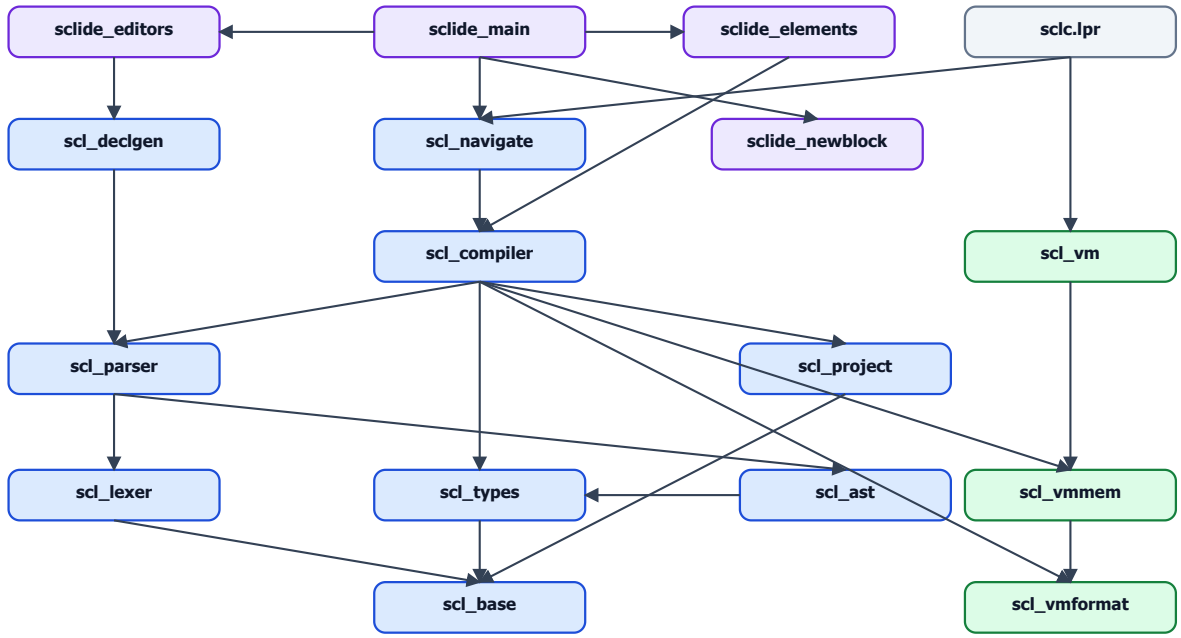
- แยก core (ไม่มี GUI) ออกจาก IDE — core ใช้ได้ทั้ง CLI, GUI และ test
- Compile ทั้งโปรเจกต์ทุกครั้ง (เร็ว ~15–200 ms สำหรับ 14 block) จึงไม่มี incremental state ให้ผิดพลาด
- ส่วนที่ Virtual PLC ต้องใช้ (scl_vmformat, scl_vmmem, scl_vm) ไม่ขึ้นกับ compiler
- Diagnostics สะสมใน TDiagList: parser กู้คืนจาก error ที่ระดับ statement / block เพื่อรายงานหลาย error พร้อมกัน
- Address ทุกตัวเป็นหน่วย bit ใน Int64 เดียว → Bool, array of Bool และ struct ใช้กลไกเดียวกัน
- Code generation แบบ stack machine: ไม่มี register allocation, ง่ายต่อการทำ VM

ลำดับขั้นตอน

ขั้น	method	รายละเอียด
1	ParseSources	อ่านทุกไฟล์ใน [Sources] (หรือ buffer จาก editor ผ่าน OnGetSource) → TSourceUnitList
2	RegisterBlocks	สร้าง TBlockInfo ต่อ block, ตรวจชื่อซ้ำ, optimized access ตาม CPU
3	AssignNumbers	เลข block จาก [BlockNumbers], ชื่อแบบ DB10/FC5, ชื่อ OB มาตรฐาน (Main=1, Startup=100), อัตโนมัติ; event class ของ OB
4	RegisterTags	tag CSV → address (MakeAddr) + type, ตรวจขนาด address/type, user constant
5	ResolveBlock	สร้าง TSclType ของ UDT/DB/FB instance/FC param/temp, layout, local symbol table (recursive พร้อมตรวจ cycle)
6	BuildDBImage	start value จาก declaration + BEGIN, retain map
7	GenerateBlock	type check + emit bytecode ต่อ block, line table, prologue สำหรับ String temp
8	BuildImage	รวมเป็น TSvImage: block table, DB, tags, vars table, constant pool

ชั้น	method	รายละเอียด
9	SaveImage / ListingText	เขียน .svpl, .lst (disassembly), .map.txt

โครงสร้าง unit (dependency)



สีเขียว = ใช้ร่วมกับ Virtual PLC ได้ (ไม่มี dependency กับ compiler) · สีม่วง = GUI (LCL + SynEdit) · ลูกศร = uses

รูป 2.2 Unit และ dependency

หน้าที่ของแต่ละ unit

Unit	บรรทัด	หน้าที่
scl_base	156	TCpuFamily, TSrcPos, TDiag/TDiagList, ESclError
scl_types	785	TTypeKind, TSclType, TMember, layout S7, system FB types, TConstValue
scl_lexer	668	token, typed constant (T#, 16#, D#, TOD#...), address %, pragma {...}, comment
scl_ast	643	AST: declaration, expression, statement, TProgUnit
scl_parser	1,424	recursive descent parser + error recovery
scl_project	351	.sclproj (INI), tag CSV, block numbers
scl_compiler	4,396	semantic analysis, layout, DB image, code generation, image building, listing
scl_declgen	354	grid ⇄ source ของ interface, template block ใหม่
scl_navigate	345	Find Declaration
scl_vmformat	1,003	opcode, TCodeBuf, TSvImage (อ่าน/เขียน .svpl), disassembler
scl_vmmem	279	encode/decode ค่าใน memory (big endian, BCD, S5TIME, DT, String)
scl_vm	948	reference Virtual PLC interpreter
sclc.lpr	394	CLI
sclide_main	1,884	หน้าต่างหลัก, menu, tree, compile, find declaration, recent, panels
sclide_editors	1,007	แท็บ editor: source, tags, interface grid, grid view, text

Unit	บรรทัด	หน้าที่
sclide_elements	613	panel SCL elements + drag & drop
sclide_newblock	185	dialog สร้าง block

โครงสร้างข้อมูลหลัก

ชนิด	อยู่ที่	ความหมาย
TProgUnit	scl_ast	block 1 ตัวจาก source: header, sections, body, ตำแหน่งใน text
TBlockInfo	scl_compiler	ข้อมูลหลัง resolve: เลข, DataType, ParamType, TempType, Locals, Image, CodeIndex, Calls
TSclType / TMember	scl_types	ชนิดข้อมูลและ member พร้อม OffsetBits, Section, ByRef, Init
TLocalSym	scl_compiler	ชื่อใน block → (instance / frame / constant, offset, ByRef, DeclPos)
TExpr.RType/RefKind/RefOfs	scl_ast	ผลการวิเคราะห์ชนิดและการเข้าถึง (cache ใน node)
TCodeBuf	scl_vmformat	buffer bytecode + label/fixup
TSvImage / TSvBlock / TSvDataBlock	scl_vmformat	image ใน memory ก่อนเขียนไฟล์ / หลังอ่านไฟล์

แหล่งข้อมูลของหัวข้อนี้

- โค้ด core/*.pas
- หลักการ compiler ทั่วไป: Aho, Lam, Sethi, Ullman "Compilers: Principles, Techniques, and Tools"; Robert Nystrom "Crafting Interpreters" (stack VM, single-pass codegen)

ภาษา SCL ที่รองรับ

Block และส่วนประกอบ

หมวด	รองรับ
Block	TYPE (UDT), DATA_BLOCK (STRUCT / VAR / instance ของ FB, UDT, system FB), FUNCTION, FUNCTION_BLOCK, ORGANIZATION_BLOCK
Header	TITLE, VERSION, AUTHOR, FAMILY, NAME, { S7_Optimized_Access := ... }, RETAIN / NON_RETAIN, KNOW_HOW_PROTECT (ข้าม)
Section	VAR_INPUT, VAR_OUTPUT, VAR_IN_OUT, VAR (RETAIN/NON_RETAIN), VAR_TEMP, VAR CONSTANT
Statement	:=, IF/ELSIF/ELSE, CASE (ค่า, รายการ, ช่วง a..b, ELSE), FOR..TO..BY, WHILE, REPEAT..UNTIL, EXIT, CONTINUE, RETURN, REGION
Operator	OR XOR AND & = <> < <= > >= + - * / MOD ** NOT, slice .%X .%B .%W .%D
Call	FC (named/positional, => output, InOut), FB multi-instance #i(...), single instance "iDB"(), system FB, IEC_TIMER.TON(...)
Constant	16#FF 2#1010 8#17, INT#5, W#16#.., T#1s500ms, LT#, S5T#, D#, TOD#, DT#, LDT#, string literal + escape $, array init [1, 2, 3(0)]
Address	%I0.0 %IW64 %QX0.1 %MW10 %DB10.DBX0.0

ชนิดข้อมูล

ชนิด	บิต	S7-300/400	S7-1200	S7-1500
Bool, Byte, Word, DWord, Int, DInt, Real, Time, Date, TOD, Char, String	1–2048	✓	✓	✓
S5Time, DT (Date_And_Time)	16/64	✓	S5Time ✗ / DT ✓	✓
SInt, USInt, UInt, UDInt, LReal, WChar, WString, DTL	8–64	✗	✓	✓
LInt, ULInt, LWord, LTime, LTOD, LDT	64	✗	✗	✓
Array (หลายมิติ), Struct, UDT, FB instance	-	✓	✓	✓

Function / instruction

หมวด	รายการ
Conversion	X_TO_Y ทุกคู่ของตัวเลข/บิต/เวลา, TRUNC, ROUND, CEIL, FLOOR, NORM_X, SCALE_X
Math	ABS, SQR, SQRT, EXP, LN, LOG, SIN, COS, TAN, ASIN, ACOS, ATAN, FRAC
Select / compare	MIN, MAX, LIMIT, SEL, MUX
Bit	SHL, SHR, ROL, ROR, slice
String	LEN, CONCAT, LEFT, RIGHT, MID, FIND, เปรียบเทียบ, กำหนดค่า
System FB	TP, TON, TOF, TONR (+_TIME, _LTIME, IEC_TIMER), CTU, CTD, CTUD (+_INT, _DINT, _UDINT, IEC_COUNTER), R_TRIG, F_TRIG, RS, SR

กฎที่ตรวจตาม CPU

- S7-300/400: ไม่มี optimized access (เตือนและใช้ standard), ไม่มี slice access, ไม่มีการแปลง bit string ⇄ integer แบบ implicit, OB ต้องระบุเลข

- S7-1200: ไม่มีชนิด 64-bit integer/time (ยกเว้น LReal), ไม่มี S5Time, array index ต้องอยู่ใน -32768..32767
- S7-1500: รองรับทุกชนิด รวม *_LTIME
- Implicit conversion: อนุญาตเฉพาะขยายขนาด (Int→DInt→LInt, Int→Real, Real→LReal, Byte→Word...) ที่เหลือต้องใช้ X_TO_Y
- ค่าคงที่ตรวจช่วง (เช่น Int := 70000 → error), index คงที่ตรวจ bounds ตอน compile, index แปรผันตรวจตอน runtime

ยังไม่รองรับ

- ANY, POINTER, VARIANT, REF_TO, DB_ANY, Array[*]
- AT overlay, GOTO / label
- ฟังก์ชันแปลง String (INT_TO_STRING, STRING_TO_INT ...)
- S5 timer / counter (T, Z area) ของ S7-300/400
- System function ของ CPU (RD_SYS_T, PEEK/POKE, communication, ProDiag)
- EN/ENO (ละไว้)
- LAD/FBD/STL/GRAPH (รองรับเฉพาะ SCL)

แหล่งข้อมูลของหัวข้อนี้

- โค้ด scl_types (TypeSupportedOn), scl_compiler (ImplicitOK, InferBuiltin)
- ไฟล์ทดสอบ examples/FeatureTest, examples/ErrorTest
- ความรู้เรื่องชนิดข้อมูลและ instruction ของ S7 — ควรตรวจกับ TIA Portal Information System หัวข้อ "Data types" และ "SCL instructions"

รายละเอียดภายใน Compiler

Lexer (scl_lexer)

- ตัด token ทั้งไฟล์ล่วงหน้า (TokenizeAll) → parser มองไปข้างหน้าได้หลาย token
- ชื่อ: plain, #local, "global"; keyword เป็น ident แล้ว parser ตัดสินเอง (เพราะชื่ออย่าง IN, MIN, VALUE ใช้เป็น parameter ได้)
- Typed constant: prefix + # → แปลงเป็นค่า (Time = ms, LTime = ns, Date = วันนับจาก 1990-01-01, DT = ms นับจาก 1990)
- Address % อ่านรวมจุด เช่น %DB10.DBX0.0, slice .%X0 ใช้ token เดียวกัน
- Pragma {...} เก็บ text ไว้ parse เป็น key := value

Parser (scl_parser)

- Recursive descent; precedence: OR < XOR < AND < = <> < < <= > >= < + - < * / MOD < ** < unary < postfix
- REGION: ชื่อคือข้อความที่เหลือของบรรทัด
- CASE: แยก label กับ statement ด้วย lookahead (ค่า/ชื่อ ตามด้วย : , ..)
- Error recovery: ข้ามไปที่ ; หรือ END_xxx ของ block แล้วทำต่อ
- เก็บตำแหน่ง HeaderEndOffset / DeclEndOffset เพื่อให้ grid editor แทนที่ส่วน declaration ได้
- เก็บ raw text ของ type และ initial value (TypeText / InitText) เพื่อ round-trip

Type system และ layout (scl_types)

- TSclType ต่อชนิด; elementary เป็น singleton, composite อยู่ใน type pool (ล้างทุกครั้งที่ compile)
- Layout แบบ S7 standard: Bool เรียง bit, Byte เริ่ม byte ถัดไป, ชนิดอื่น/Array/Struct/String เริ่ม byte คู่, Struct ปัดเป็น byte คู่
- FB instance: [Input][Output][InOut=Ptr][Static]; FC param: [RET_VAL][Input][Output][InOut]; temp ต่อท้าย
- System FB: layout คงที่ (ดูหัวข้อ 06) เพื่อให้ VM รู้ offset

Semantic analysis

- InferType(E) คำนวณชนิด + constant folding และ cache ใน node (RType, IsConst, CVal, RefKind, RefOfs, ByRef)
- ชื่อ: local ก่อน (#) แล้ว tag, DB, block; builtin ตรวจเมื่อเรียกแบบ function และชื่อไม่ถูก local ที่เป็น instance บัง
- CommonType: ชนิดผลของ operator (literal ปรับตามอีกฝั่ง, signed/unsigned ผสม → signed ที่ใหญ่กว่า, int+real → real)
- ImplicitOK: กฎ implicit conversion (ดูหัวข้อ 03)
- ตรวจ argument ของ FC/FB: ชื่อ parameter, ทิศทาง (:= / =>), InOut ต้องเป็นตัวแปร, FC ต้องกำหนด input ครบ

Code generation

รูปแบบ	bytecode ที่ได้
#a := #b + 1 (Int)	AddrLocal a · AddrLocal b · Load Int · PushI 1 · AddI · Wrap Int · Store Int
IF c THEN .. ELSE .. END_IF	eval c · Jz Lelse · ... · Jmp Lend · Lelse: ... · Lend:
CASE x OF 1: .. 4..9: ..	เก็บ x ใน temp ซ่อน · เทียบทีละ label → Jnz body
FOR i := a TO b BY s	temp ซ่อนเก็บ b (และ s ถ้าไม่คงที่) · ตรวจ overflow ก่อนบวก

รูปแบบ	bytecode ที่ได้
FC call	FrameAlloc · เขียน param ด้วย AddrNewFrame · Call · อ่าน output/RET_VAL · FrameFree
FB call	เขียน input ลง instance · push instance · CallFB · copy output
slice w.%X3 := b	AddrX w · Dup · Load · (value) · SetBit 3 · Store
String	ค่าคือ address; literal อยู่ใน constant pool; CONCAT ใช้ temp String[254]

- ทุก block เริ่มด้วย Jmp ไป prologue (ตั้ง max length ของ String temp) แล้วกลับมา
- Line table: (code offset → บรรทัด) สำหรับ online monitor ในอนาคต
- Integer ทุกชนิดคำนวณเป็น Int64 แล้ว Wrap ตามชนิด (overflow แบบ S7), Real คำนวณ Double แล้ว F2F32

DB image และ retain

- ApplyDefaults: ค่าเริ่มต้นจาก declaration (รวม instance ซ้อนและ system FB), ตั้ง max length ของ String ทุกตัว
- BEGIN ของ DB: path := ค่าคงที่ (StaticPath คำนวณ offset)
- Retain: DB standard ที่ไม่มี keyword = retain ทั้ง block (instance DB = non-retain) · optimized: ตาม VAR RETAIN · retainMap 1 bit ต่อ byte

เลข block และ OB

- ลำดับ: [BlockNumbers] ในโปรเจกต์ → ชื่อแบบ DB10/FC3 → ชื่อ OB มาตรฐาน → อัตโนมัติ (OB ≥ 123 สำหรับ 1200/1500)
- Event class: 1 cycle, 10 time-of-day, 20 delay, 30 cyclic (interval จาก [CyclicInterval] หรือค่ามาตรฐาน OB30..38), 40 HW, 80.., 100 startup, 121/122 error

Declaration round-trip (scl_declgen) และ Find Declaration (scl_navigate)

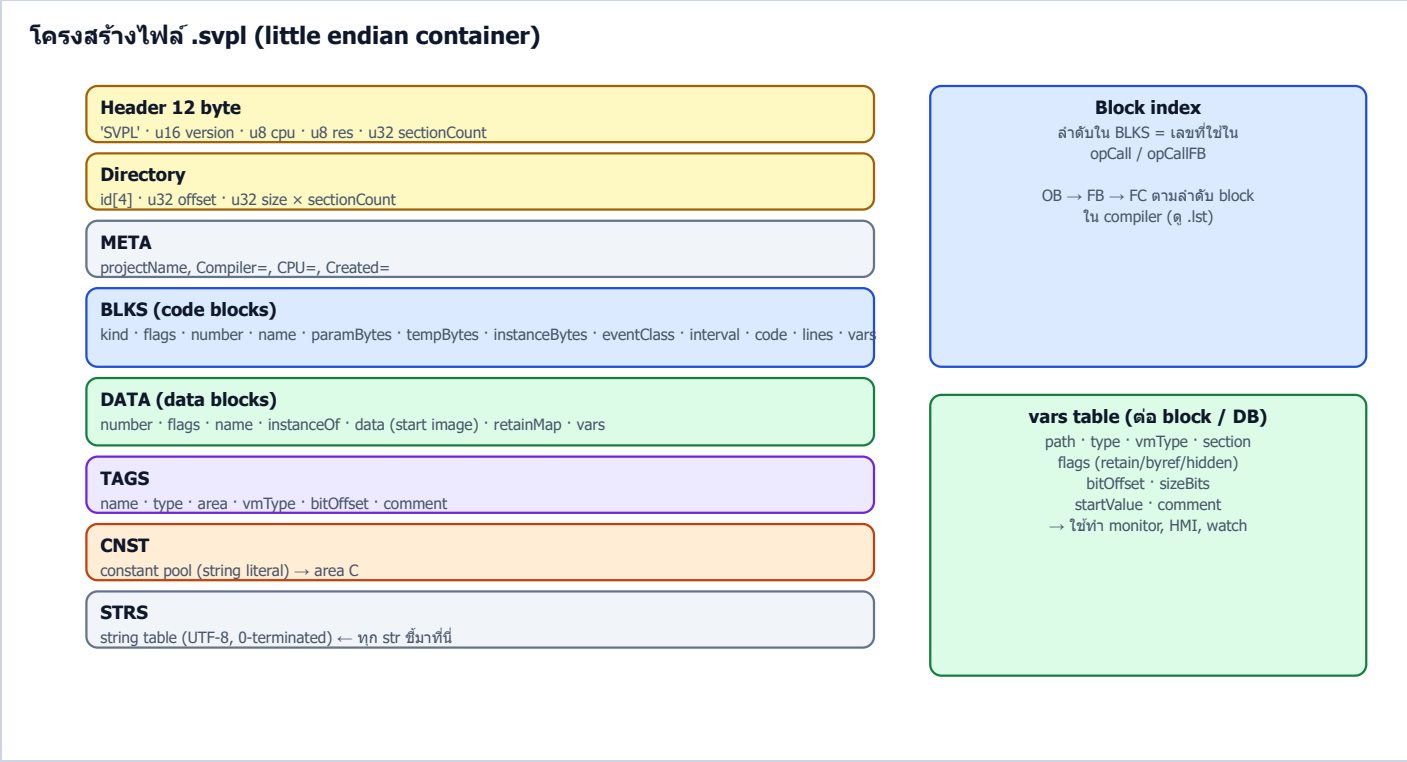
- LoadDeclRows: parse source → แถว (section, name, type text, init, retain, comment)
- ApplyDeclRows: สร้าง VAR sections / STRUCT ใหม่แล้วแทนที่ช่วง HeaderEnd..DeclEnd ของ block นั้น
- FindDeclarationAt: แยก path ใต้ cursor (#a.b[i].c) → local / DB member / tag / block → ไล่ member ทีละชั้นด้วย TSclType → DeclPos

---

แหล่งข้อมูลของหัวข้อนี้

- โค้ด core/*.pas
- กฎ layout standard access ของ S7 — ตรวจกับ TIA Portal Information System หัวข้อ "Memory areas / Data storage in standard access blocks"
- การตรวจ layout จริง: DB_HMI 82 byte เทียบกับ register map ของ FluffSilo HMI (จากโฟลเดอร์ PLC_S7-1200)

รูปแบบไฟล์ .svpl และ Memory model



รูป 5.1 โครงสร้างไฟล์ .svpl

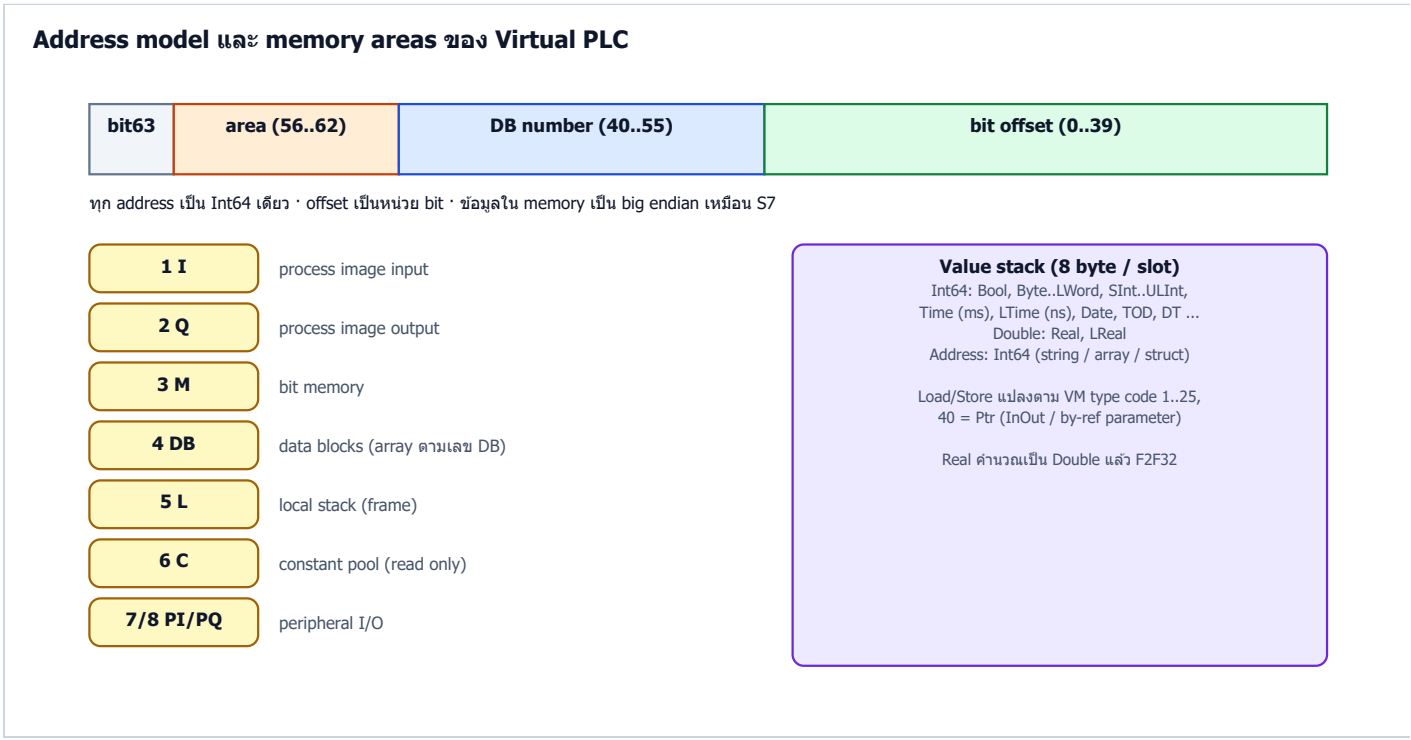
Header และ directory

offset	ขนาด	ฟิลด์
0	4	magic = 'SVPL'
4	2	version = 1
6	1	cpu: 0 S7-300, 1 S7-400, 2 S7-1200, 3 S7-1500
7	1	reserved
8	4	section count
12	12×n	directory: id[4], u32 offset, u32 size

Sections

id	เนื้อหา
META	str projectName, u32 n, str × n (Compiler=, CPU=, Created=)
BLKS	u32 count; ต่อ block: u8 kind(1 OB/2 FC/3 FB), u8 flags, u16 number, str name, str sourceFile, u32 paramBytes, u32 tempBytes, u32 instanceBytes, u16 eventClass, u16 res, u32 interval, bytes code, u32 nLines + (u32 ofs, u32 line)×n, vars
DATA	u32 count; ต่อ DB: u16 number, u8 flags(1 opt,2 retain,4 instance), u8 res, str name, str instanceOf, bytes data, bytes retainMap, vars
TAGS	u32 count; str name, str type, u8 area, u8 vmType, u16 res, i64 bitOffset, str comment
CNST	bytes constant pool (String literal รูปแบบ S7)
STRS	string table UTF-8 ปิดท้ายด้วย 0 — str = u32 offset ในตารางนี้

vars: u32 count; ต่อรายการ str path, str type, u8 vmType (0 = aggregate), u8 section, u8 flags (1 retain, 2 byref, 4 hidden, 8 array, 16 struct), u8 res, i64 bitOffset, i64 sizeBits, str startValue, str comment



รูป 5.2 Address model และ memory areas

VM type code (Load/Store)

code	ชนิด	ในหน่วยความจำ	บน stack
1	Bool	1 bit	0/1
2–5	Byte Word DWord LWord	1/2/4/8 byte	unsigned
6–9	SInt Int DInt LInt	two complement	signed
10–13	USInt UInt UDInt ULInt		unsigned
14/15	Real / LReal	IEEE-754	Double
16/17	Time / LTime	DInt ms / LInt ns	Int64
18	S5Time	BCD + time base	ms
19–21	Date TOD LTOD	UInt / UDInt / ULInt	วัน / ms / ns
22	DT	BCD 8 byte	ms นับจาก 1990
23	LDT	LInt	ns นับจาก 1970
24/25	Char / WChar	1 / 2 byte	code
40	Ptr	8 byte raw address	address

> ข้อเสนอสำหรับ version 2: เพิ่ม layout hash ต่อ DB ใน section DATA เพื่อให้ Virtual PLC รู้ว่าจะคงค่า retain ข้าม download ได้หรือไม่ และเพิ่ม hash ของทั้งไฟล์ใน META เพื่อเทียบ online/offline

---

แหล่งข้อมูลของหัวข้อนี้

- core/scl_vmformat.pas (SaveToStream/LoadFromStream), core/scl_vmmem.pas
- docs/SVPL_Spec.md

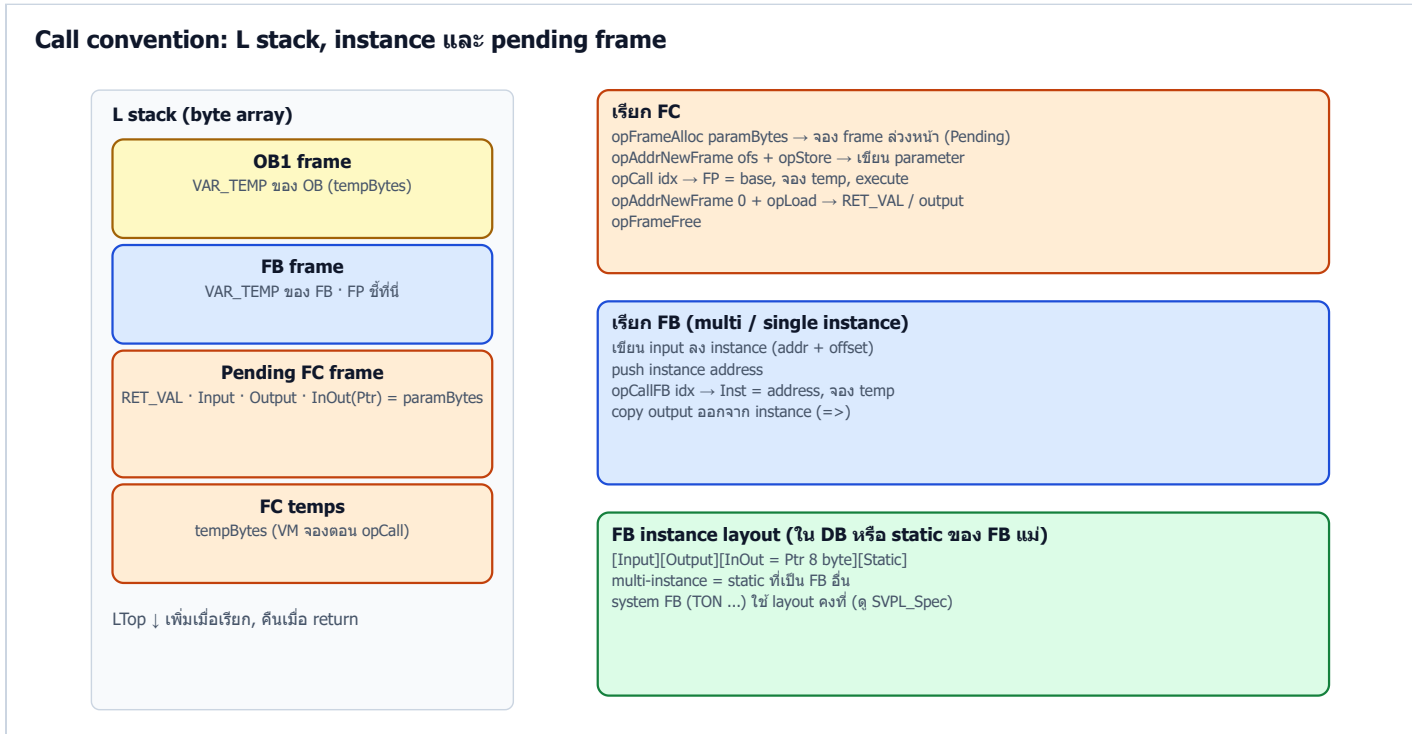
- รูปแบบ BCD ของ S5TIME / DATE_AND_TIME — ตรวจกับเอกสาร Siemens "Data types" ของ S7-300/400

Instruction set และ Call convention

operand: b=u8 w=u16 i=i32 u=u32 q=i64 f=f64 (little endian หลัง opcode 1 byte) · jump offset นับจากต้นคำสั่งถัดไป

#	Opcode	Operand	Stack	ความหมาย
0	Nop			
1	PushI	q	→ v	ค่าคงที่ integer
2	PushF	f	→ f	ค่าคงที่ double
3	PushAddr	q	→ a	address absolute
4	AddrInst	u	→ a	Inst + bits
5	AddrLocal	u	→ a	L: FP*8 + bits
6	AddrNewFrame	u	→ a	L: pending frame + bits
7	AddrAdd	u	a → a	+ bits
8	AddrIndex	i lo, u stride, u count	a i → a	index + range check
9	Load	b type	a → v	
10	Store	b type	a v →	
11	LoadPtr		a → a	อ่าน pointer 8 byte
12	MemCpy	u bits	dst src →	
13	StrCpy	b wide	dst src →	ตัดตาม max ของ dst
14–16	Dup Drop Swap			
17–23	AddI SubI MulI DivI ModI NegI AbsI			Int64; หาร 0 → 0
24–30	AddF SubF MulF DivF NegF ExptF AbsF			Double
31–34	And Or Xor Not			bitwise
35–44	EqI..GeI, LtU..GeU		a b → bool	signed / unsigned
45–50	EqF..GeF			
51	Wrap	b type	v → v	ตัดตามความกว้าง
52/53	I2F U2F			
54–57	F2ITrunc F2IRound F2IFloor F2ICeil			Round = banker
58	F2F32			single precision
59/60	GetBit SetBit	b n		slice X
61/62	GetField SetField	b sh, b w		slice B/W/D

#	Opcode	Operand	Stack	ความหมาย
63–66	Shl Shr Rol Ror	b width	v n → v	
67–70	MinI MaxI MinF MaxF			
71	Sel		g a b → r	
72–74	Jmp Jz Jnz	i		
75	Call	w block		FC
76	CallFB	w block	a →	FB
77/78	FrameAlloc FrameFree	u / -		
79	Ret			
80	Builtin	w id	f → f	SQRT EXP LN LOG SIN COS TAN ASIN ACOS ATAN FRAC
81	SysFB	w id, b cntType	a →	timer/counter/edge
82–88	StrCmp StrLen StrConcat StrLeft StrRight StrMid StrFind	b wide		
89	Trap	w code		runtime error
90	MuxPick	b n	v0..vn-1 k → vk	



รูป 6.1 L stack, pending frame และ FB instance

System FB layout (bit offset)

FB	layout
Timer (Time) TP/TON/TOF/TONR, *_TIME, IEC_TIMER — 22 byte	IN 0, R 1, PT 16, Q 48, ET 64, _ST 96 (Byte), _T0 112 (LInt)

FB	layout
Timer (LTime) *_LTIME, IEC_LTIMER — 30 byte	IN 0, R 1, PT 16, Q 80, ET 96, _ST 160, _T0 176
Counter CTU/CTD/CTUD	CU 0, CD 1, R 2, LD 3, Q 4, QU 5, QD 6, _CUO 7, _CDO 8, PV 16, CV 16+bits(type)
R_TRIG / F_TRIG	CLK 0, Q 1, _M 2
RS / SR	S 0, R1 1, Q1 2 / S1 0, R 1, Q1 2

SysFB id: 1 TP, 2 TON, 3 TOF, 4 TONR, 11–14 แบบ LTime, 20 CTU, 21 CTD, 22 CTUD, 30 R_TRIG, 31 F_TRIG, 32 RS, 33 SR

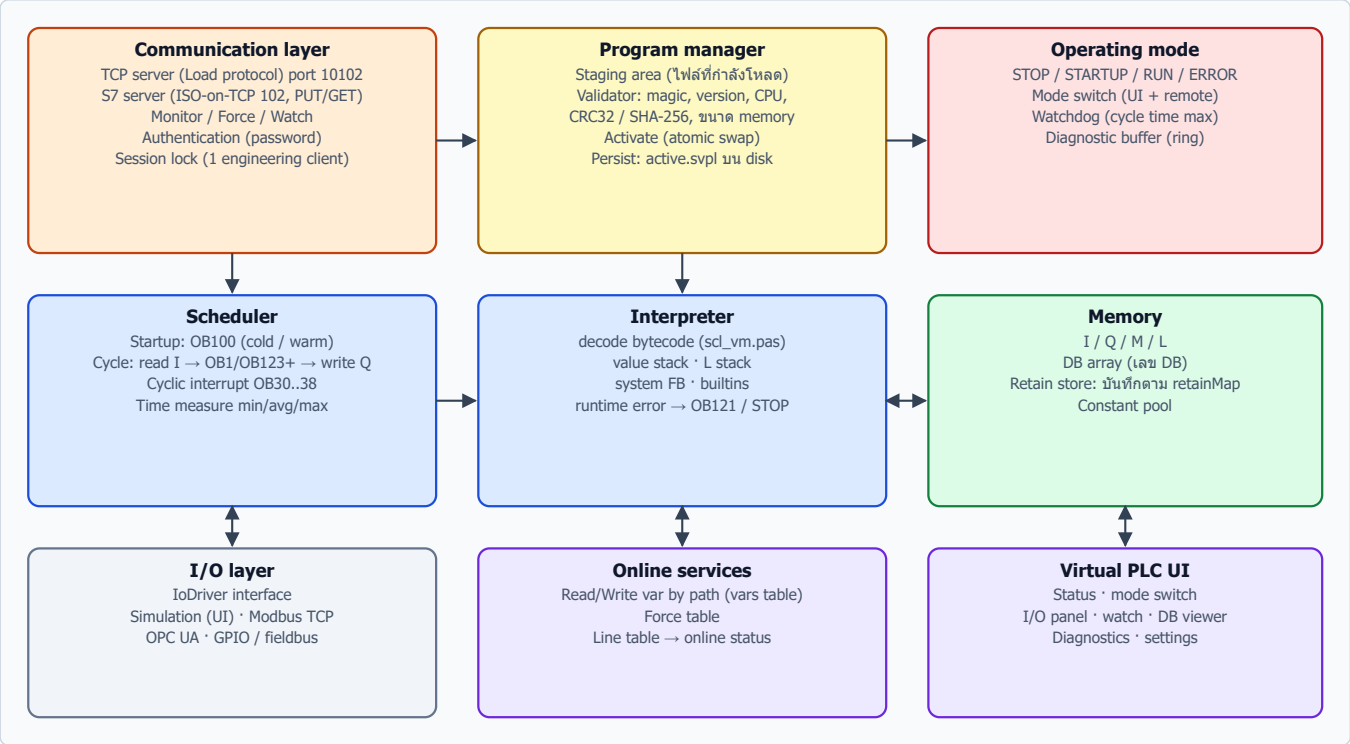
แหล่งข้อมูลของหัวข้อนี้

- core/scl_vmformat.pas (TOpcode, OpOperands), core/scl_vm.pas (Execute, SysFB)
- พฤติกรรม IEC timer/counter ตาม IEC 61131-3 ส่วน standard function blocks

โครงสร้าง Virtual PLC เพื่อรองรับการ Download

> หัวข้อนี้เป็นการออกแบบที่แนะนำ (ยังไม่ได้ implement) โดยใช้ core/scl_vm.pas ที่ทำงานได้แล้วเป็นแกนของ runtime

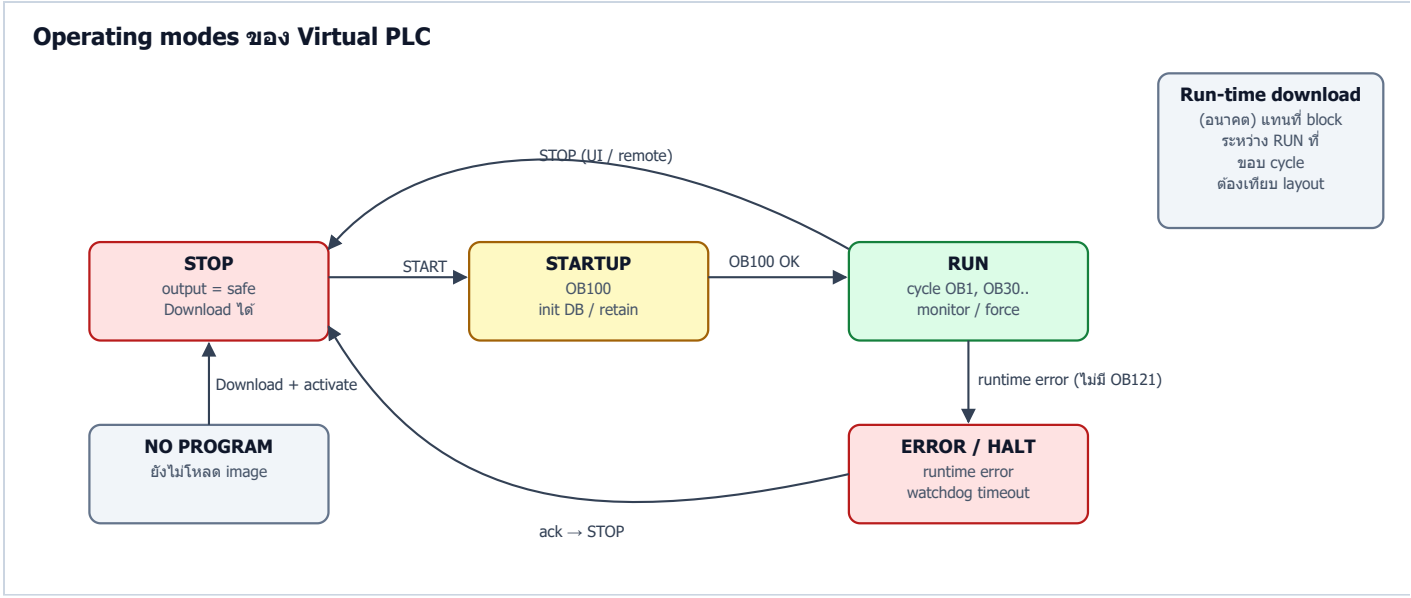
สถาปัตยกรรม Virtual PLC ที่แนะนำ (รองรับ Download)



รูป 7.1 สถาปัตยกรรม Virtual PLC ที่แนะนำ

องค์ประกอบ

ส่วน	หน้าที่	แนวทาง implement (Lazarus)
Communication layer	รับคำสั่ง Download / mode / monitor จาก IDE	fcl-net ssockets (TInetServer) หรือ Synapse, thread แยก
S7 server (ทางเลือก)	ให้ HMI เดิมอ่าน/เขียน DB ด้วย PUT/GET	Snap7 server (snap7.dll) register area DB/I/Q/M ชี้ไปที่ memory ของ VM
Program manager	staging → validate → activate, เก็บ active.svpl	TSvImage.LoadFromStream + validator
Operating mode	STOP / STARTUP / RUN / ERROR, watchdog	state machine เปลี่ยนที่ขอบ cycle เท่านั้น
Scheduler	OB100, OB1/OB123+, OB30..38, วัด cycle time	thread runtime วน loop + TEvent สำหรับ min cycle
Interpreter	รัน bytecode	ต่อยอด TVirtualPlc.Execute (เพิ่ม instruction budget สำหรับ watchdog)
Memory	I Q M DB L C + retain store	TBytes ต่อ area (ห้าม reallocate ระหว่าง RUN ถ้า Snap7 ชี้อยู่)
I/O layer	แลกค่า I/Q กับโลกภายนอก	interface IIoDriver: ReadInputs, WriteOutputs, Start/Stop
Online services	อ่าน/เขียน/force ตัวแปรตาม path	ใช้ vars table + MemRead/WriteBits
Diagnostic buffer	บันทึกเหตุการณ์ (ring 500 รายการ)	mode change, download, runtime error พร้อม block/line



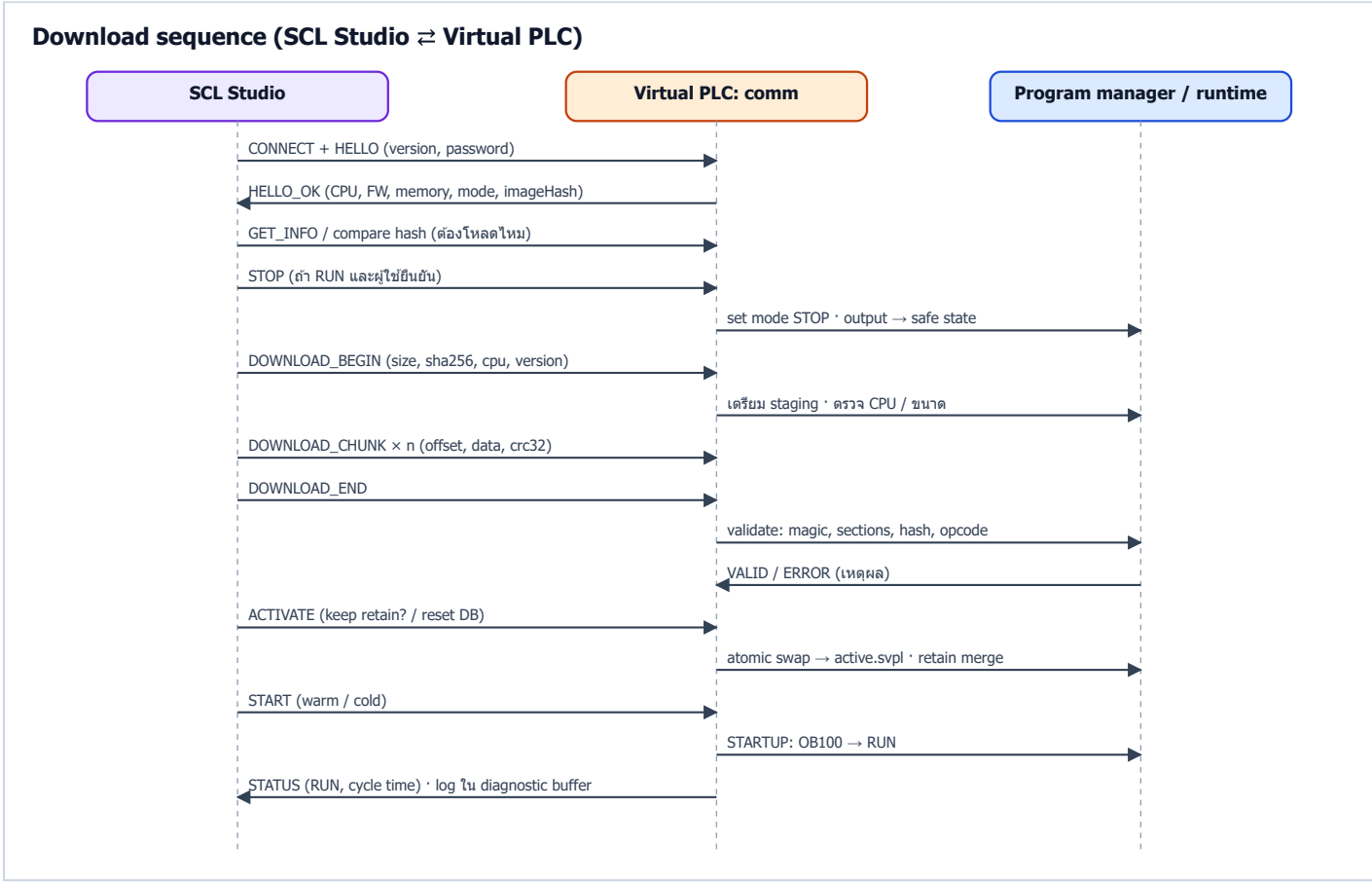
รูป 7.2 Operating modes

Load protocol (ข้อเสนอ v1, TCP port 10102)

```
Frame (little endian)
 0  u16  magic 'SV'
 2  u8   protocol version = 1
 3  u8   command / response code (response = command | 0x80, error = 0x7F)
 4  u32  sequence number (response คืนค่าเดียวกัน)
 8  u32  payload length n
12  n    payload
12+n u32 CRC32 (byte 0 .. 12+n-1)
```

code	command	payload คำขอ	payload ตอบ
01	HELLO	client name, protocol ver, password hash (SHA-256)	cpu, vplc version, memory limits, mode, image name, image SHA-256
02	GET_STATE	-	mode, cycle min/avg/max, last error
03	SET_MODE	0 STOP, 1 RUN warm, 2 RUN cold, 3 memory reset	mode ใหม่
10	DOWNLOAD_BEGIN	size u32, SHA-256, flags	max chunk size
11	DOWNLOAD_CHUNK	offset u32, data	received bytes
12	DOWNLOAD_END	-	VALID / รายการ error
13	ACTIVATE	flags: keep retain, reset DB start values, start after	ผล
14	DOWNLOAD_ABORT	-	-
18	UPLOAD	offset, length	ข้อมูล active.svpl
20/21	READ_VARS / WRITE_VARS	รายการ (address i64, vmType) / + value	values / status
22	FORCE	address, type, value / unforce	status
30	READ_DIAG	from index	รายการเหตุการณ์

error code	ความหมาย
1	frame / CRC ผิด
2	ไม่ได้ authenticate / password ผิด
3	mode ไม่ถูก (ต้อง STOP)
4	magic ผิด
5	version ไม่รองรับ
6	CPU family ไม่ตรง
7	ใหญ่เกิน memory
8	hash ไม่ตรง
9	bytecode ไม่ถูกต้อง
10	มี session อื่นถืออยู่
11	timeout



รูป 7.3 ลำดับการ Download

Validation ก่อน activate (ต้องทำครบ)

1. magic, version, CPU family ตรงกับที่ Virtual PLC ตั้งไว้
2. SHA-256 ของไฟล์ตรงกับ DOWNLOAD_BEGIN
3. ทุก section อ่านได้ ไม่เกินขอบไฟล์, string offset ถูกต้อง
4. ทุก block: เดิน bytecode ตาม OpOperands — opcode รู้จัก, operand ไม่เกินขอบ, jump ตกที่ต้นคำสั่ง, Call ชี้ FC, CallFB ชี้ FB, SysFB id รู้จัก, vmType ถูกต้อง
5. DB number ไม่ซ้ำ, ขนาดอยู่ในขีดจำกัด, tag อยู่ในขนาด process image

6. มี OB event class 1 อย่างน้อย 1 ตัว (เตือนถ้าไม่มี)
7. คำนวณ stack depth สูงสุดคร่าว ๆ (ทางเลือก) เพื่อกัน stack overflow

Activate และ retain

- Activate ทำได้เฉพาะ STOP; สลับ image แบบ atomic (เขียน active.svpl.new แล้ว rename)
- Keep retain: DB ที่เลข/ชื่อ/ขนาด (และ layout hash ในอนาคต) ตรงกัน → copy byte ที่ retainMap = 1 จาก memory เดิม; ไม่ตรง → ใช้ start value และบันทึกใน diagnostic buffer
- Reset DB start values: ใช้ค่าในไฟล์ทั้งหมด
- Retain store: บันทึก retain.bin (+CRC) ตอน STOP, ปิดโปรแกรม และทุก N วินาที

Runtime loop (pseudo code)

```
procedure TRuntime.Execute;            // thread
begin
  while not Terminated do
  begin
    ApplyPendingModeChange;            // จาก comm thread ผ่าน queue
    case Mode of
      pmStartup: begin RunOBs(100); Mode := pmRun; end;
      pmRun:
        begin
          T0 := Now;
          Io.ReadInputs(Mem.I);
          ApplyForces;
          RunOBs(1);                     // OB1 แล้ว OB123+ เรียงเลข
          RunCyclicOBsDue;               // OB30..38
          ApplyForces;
          Io.WriteOutputs(Mem.Q);
          Stats.Add(Now - T0);
          if Now - T0 > Watchdog then GoError('cycle time exceeded');
          WaitMinCycle(T0);
        end;
    else
      Sleep(10);
    end;
  end;
end;
```

Threading และความปลอดภัย

- Runtime 1 thread, comm 1 thread ต่อ client; การเข้าถึง memory จาก comm (monitor/HMI) ทำที่ขอบ cycle หรือใช้ critical section สั้น ๆ
- Watchdog: interpreter นับคำสั่ง ทุก 10,000 คำสั่งตรวจเวลา → ยกเลิก cycle และเข้า ERROR
- Output safe state เมื่อเข้า STOP/ERROR (ค่าเริ่มต้น = 0)
- Password + session lock สำหรับ download / write / force
- Log ทุก download (เวลา, ผู้ใช้, hash) ใน diagnostic buffer

Skeleton class (Free Pascal)

```
type
  TPlcMode = (pmNoProgram, pmStop, pmStartup, pmRun, pmError);
  IIoDriver = interface
    procedure ReadInputs(var I: TBytes);
    procedure WriteOutputs(const Q: TBytes);
  end;
  TVirtualPlcHost = class
    Image: TSvImage;          // active program (scl_vmformat)
    Vm: TVirtualPlc;          // interpreter (scl_vm) - memory + execute
    Mode: TPlcMode;
    Diag: TDiagBuffer;
    Io: IIoDriver;
    function  BeginDownload(Size: Cardinal; const Hash: TSha256): Integer;
    function  WriteChunk(Ofs: Cardinal; const Data: TBytes): Integer;
    function  EndDownload(out Errors: TStringList): Boolean;   // validate
    function  Activate(KeepRetain, ResetDB: Boolean): Boolean;
    procedure SetMode(M: TPlcMode; Cold: Boolean);
  end;
```

การทดสอบ Virtual PLC

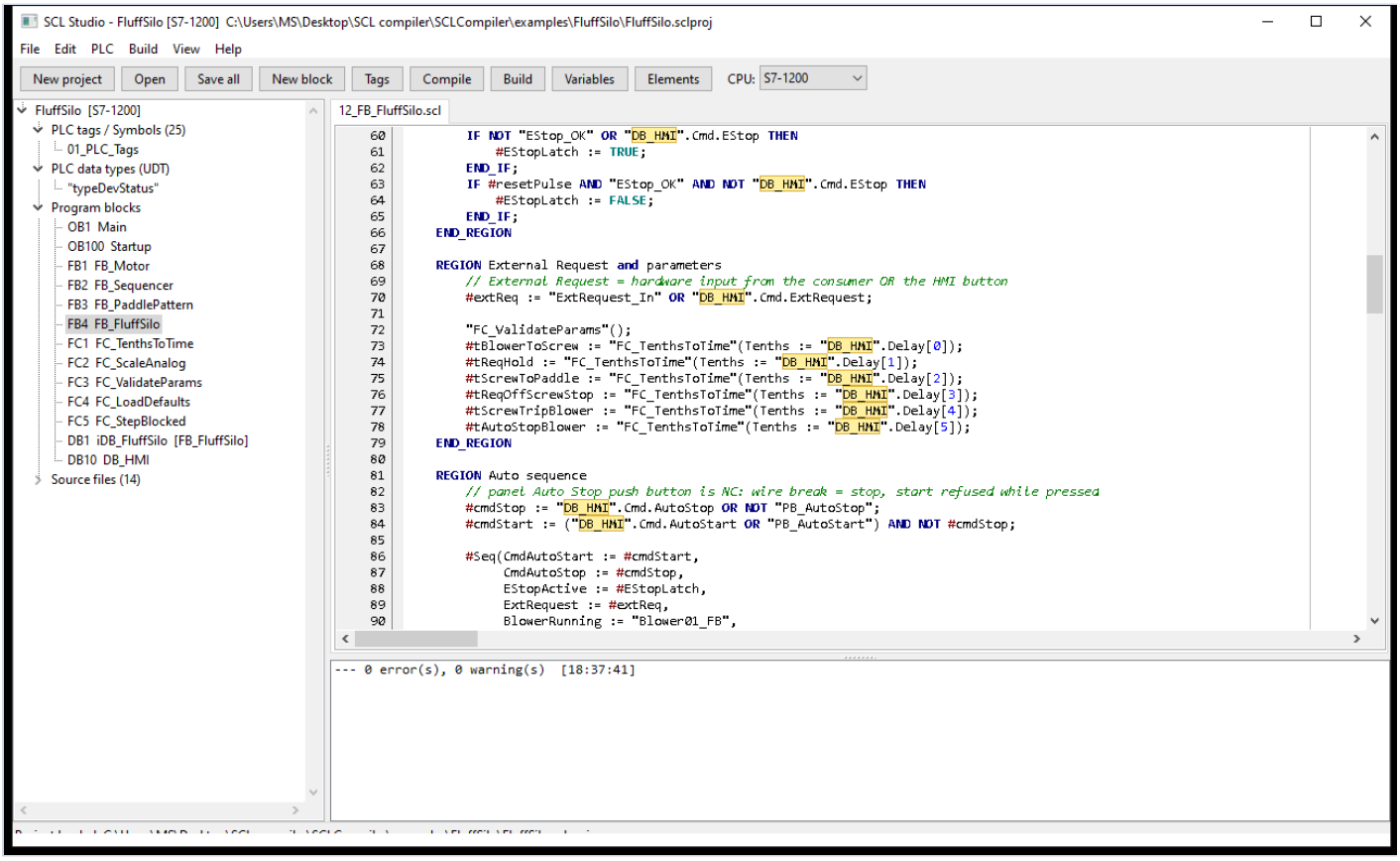
- ใช้ชุด script เดิม (tests/*.txt) รันกับ Virtual PLC จริงผ่าน protocol → ผลต้องเหมือน reference VM
- Fuzz test validator ด้วยไฟล์ .svpl ที่ถูกตัด/แก้ byte สุ่ม (ต้องไม่ crash)
- ทดสอบ download ระหว่าง RUN ต้องถูกปฏิเสธ, retain ต้องคงอยู่หลัง activate
- วัด cycle time ของ FluffSilo (เป้าหมาย < 1 ms บน PC)

แหล่งข้อมูลของหัวข้อนี้

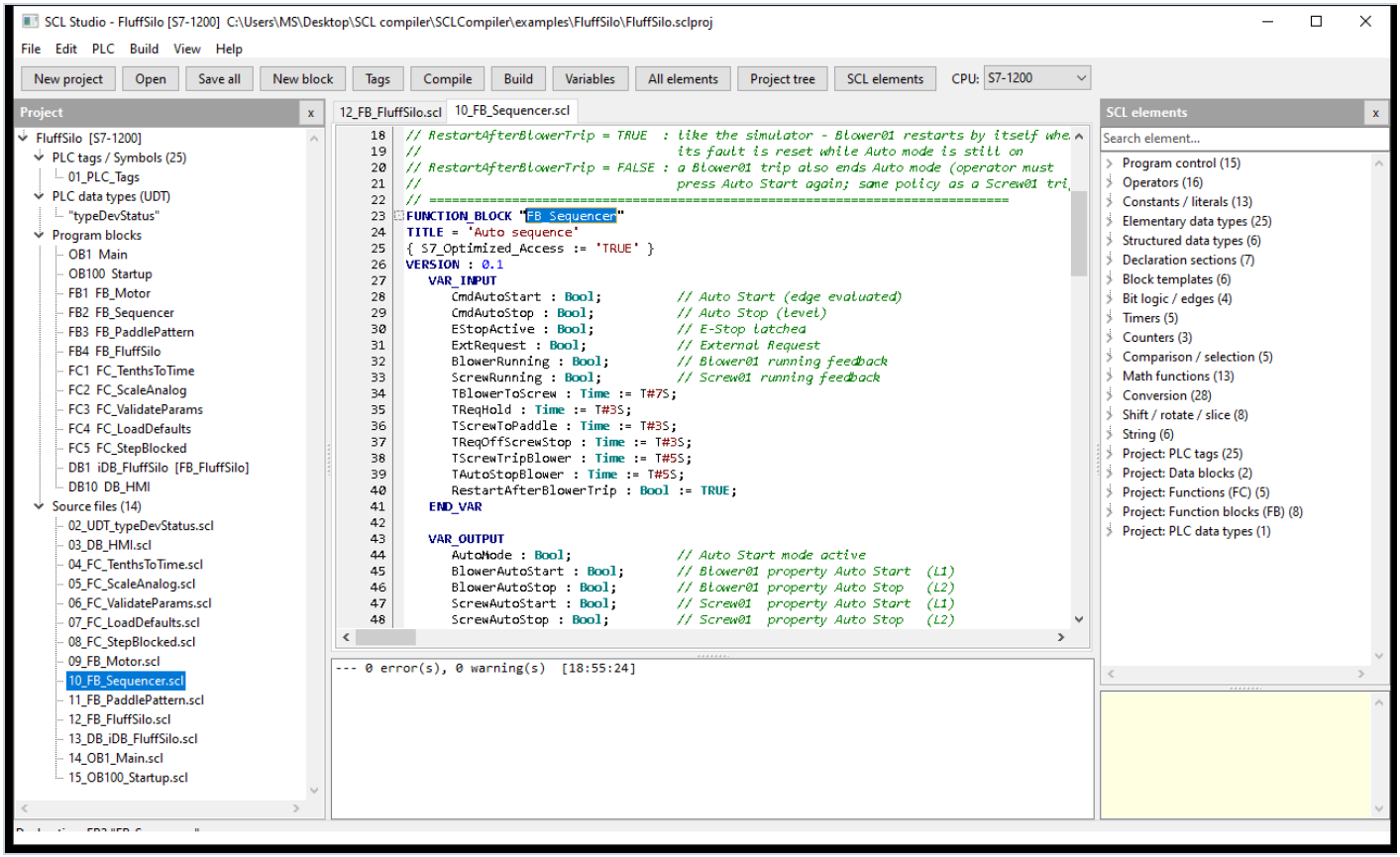
- core/scl_vm.pas (reference implementation)
- RFC 1006 (ISO transport service on top of TCP) และ ISO 8073 (COTP) — พื้นฐานของ S7 protocol port 102
- Snap7 project (snap7.sourceforge.net) — S7 server/client open source, เอกสาร S7 protocol
- Free Pascal FCL: fcl-net (ssockets), syncobjs; หรือ Synapse
- แนวคิด operating mode / OB ของ S7 — ตรวจกับ S7-1200/S7-1500 System Manual หัวข้อ "Operating modes" และ "Organization blocks"
- CRC32 (IEEE 802.3) และ SHA-256 (FIPS 180-4) สำหรับตรวจความถูกต้องของไฟล์

User Interface

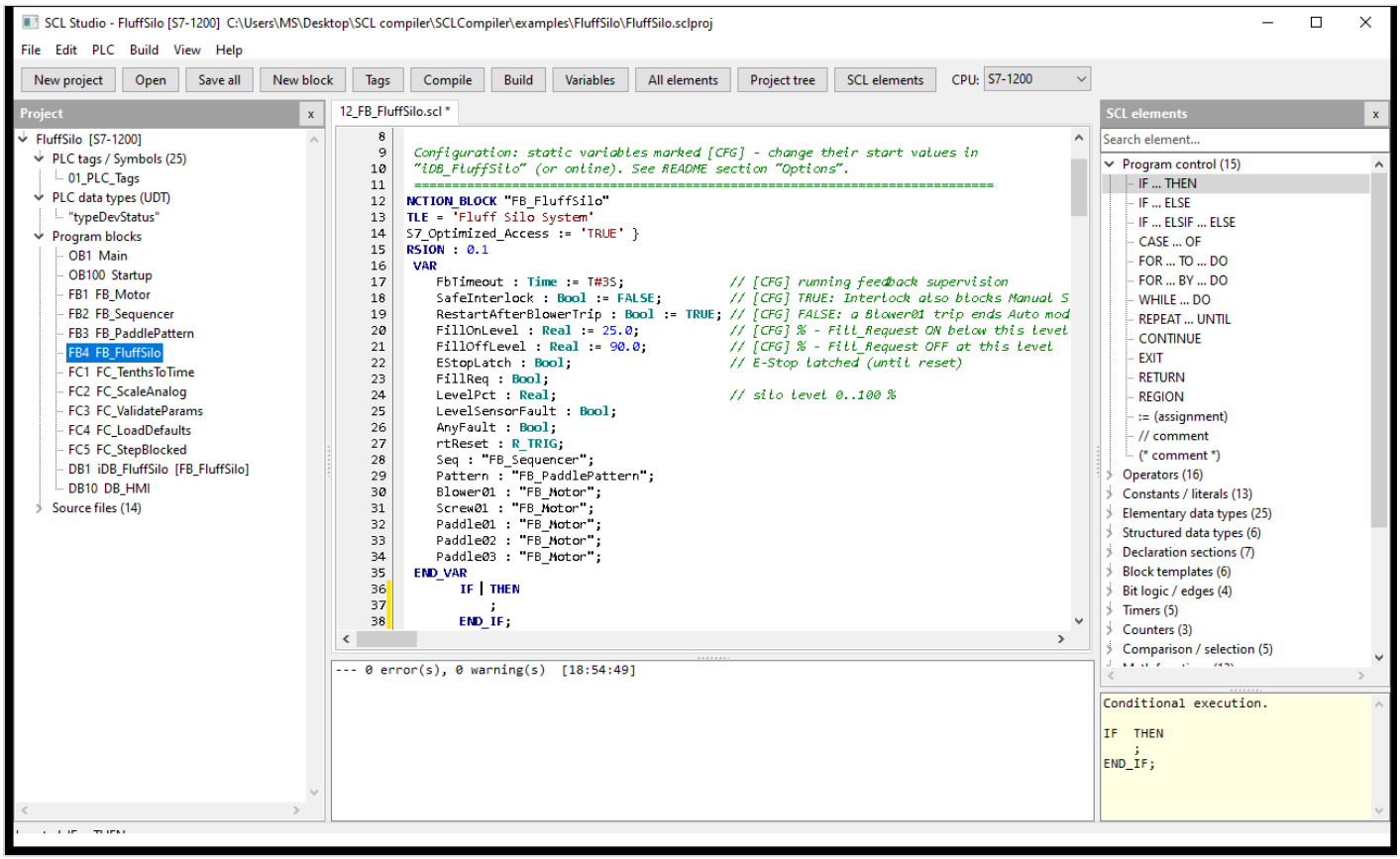
UI ปัจจุบันของ SCL Studio



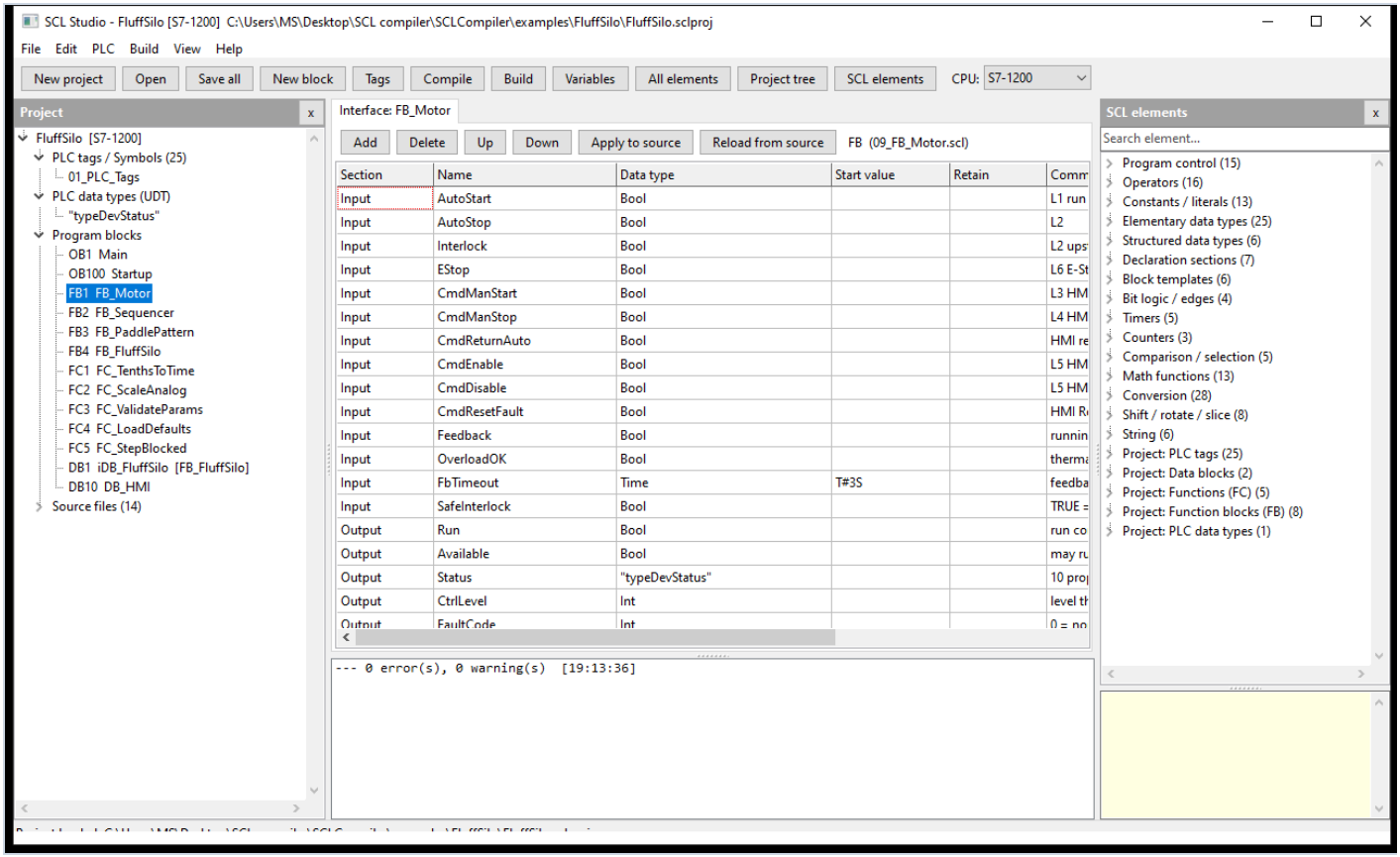
รูป 8.1 SCL editor + Auto Highlight (DB_HMI ถูกระบายทุกตำแหน่ง), Project tree ซ้าย, SCL elements ขวา



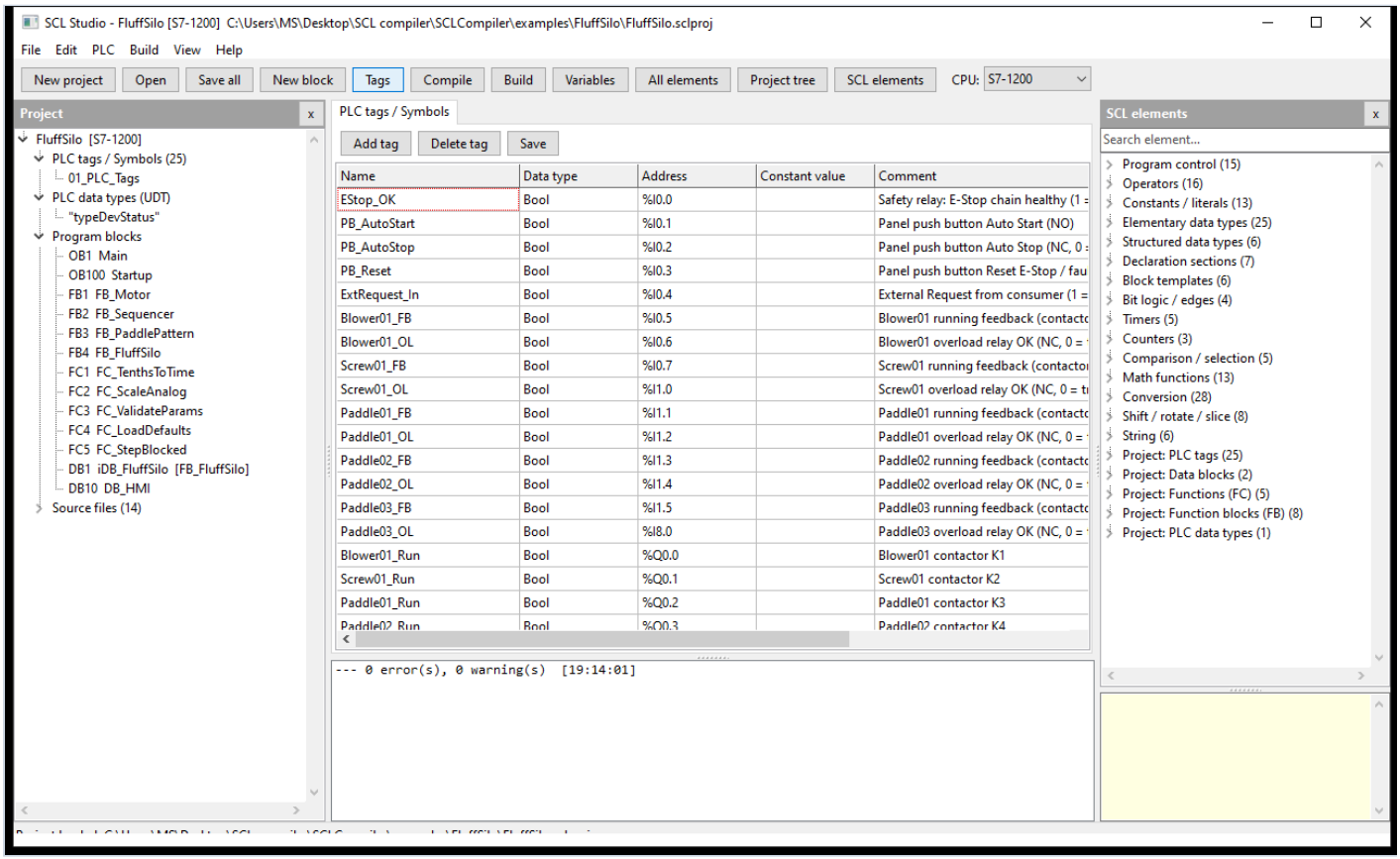
รูป 8.2 Find Declaration ข้ามไฟล์: ไฟล์ปลายทางถูกเลือกใน Project tree



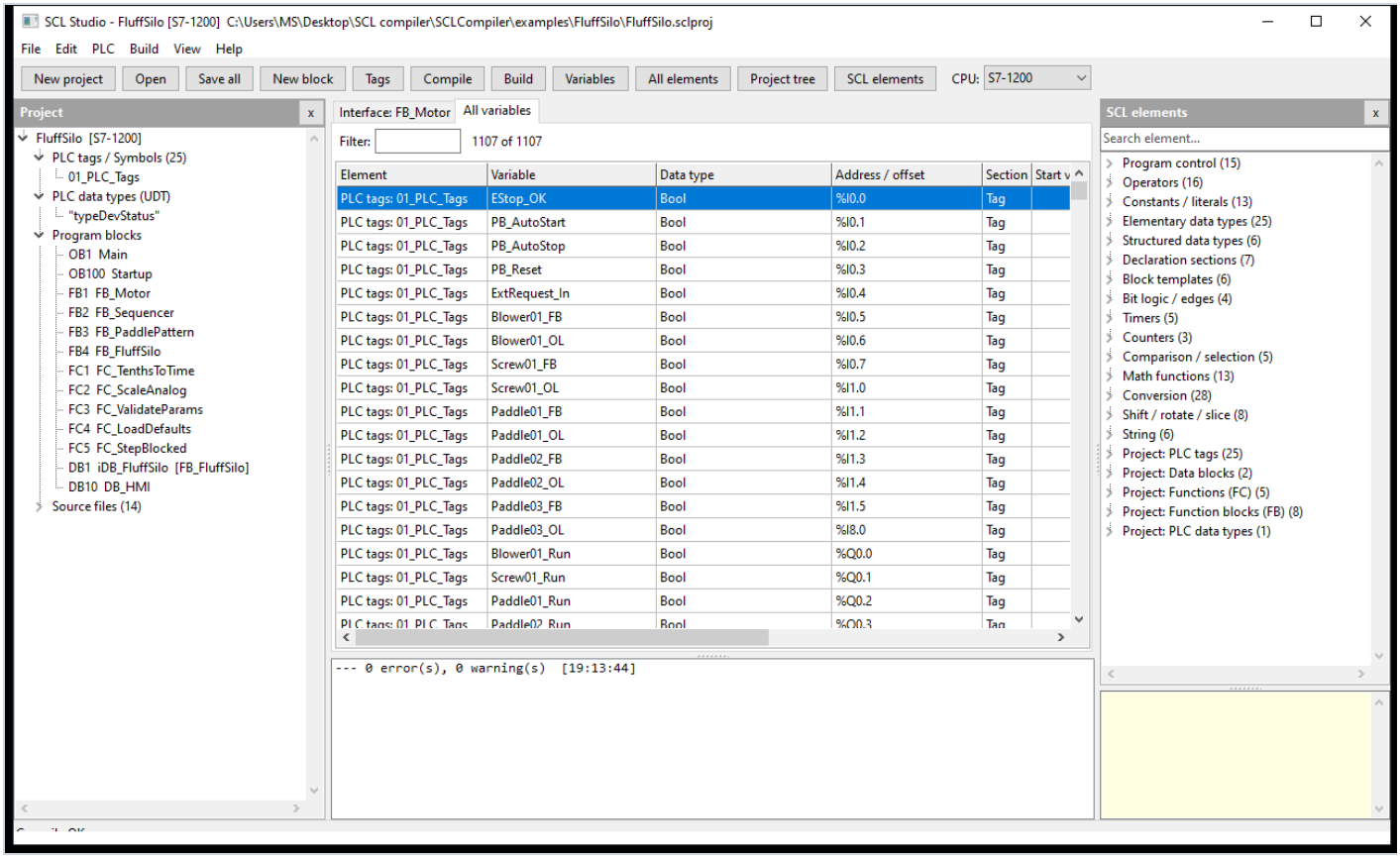
รูป 8.3 ลาก "IF ... THEN" จาก SCL elements มาวางใน editor



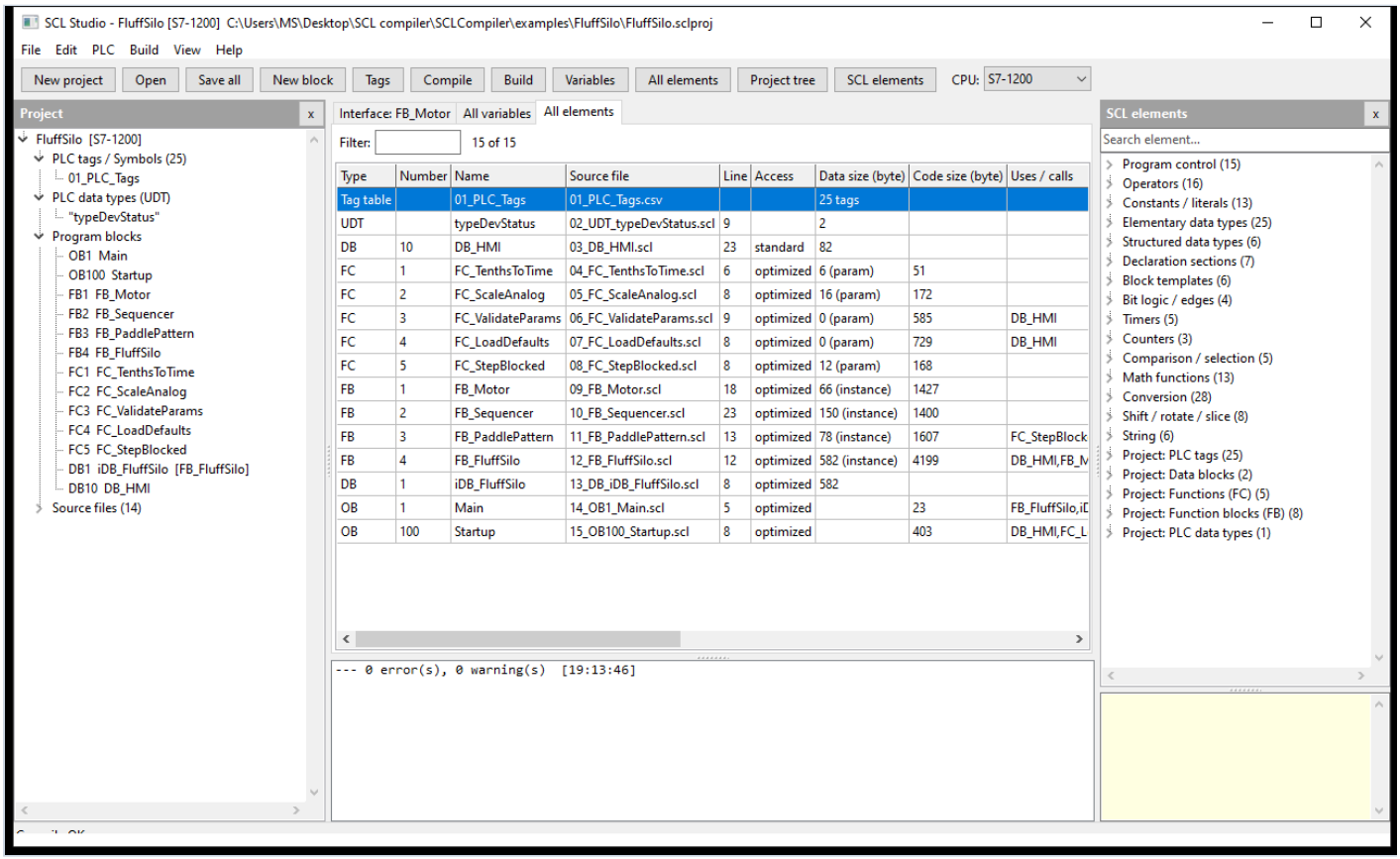
รูป 8.4 Interface editor (grid ⇄ source) ของ FB_Motor



รูป 8.5 PLC tags / Symbols



รูป 8.6 All variables (F11)

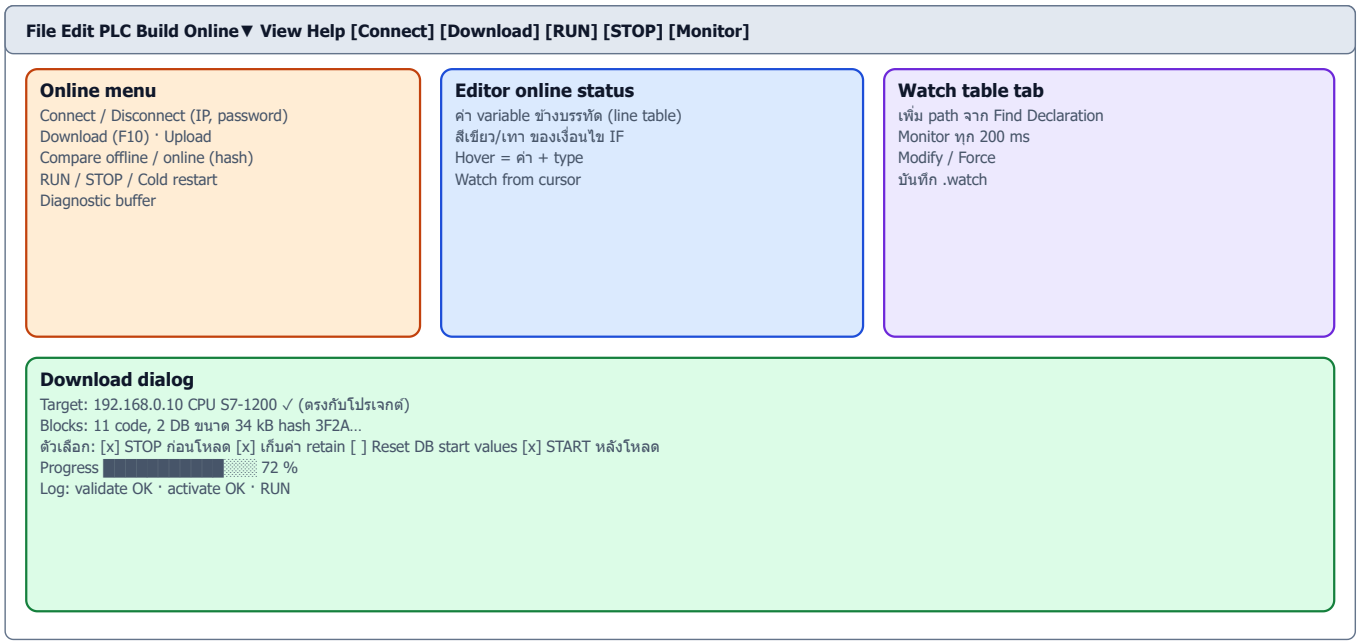


รูป 8.7 All elements (F12)

ส่วน UI	ความสามารถ
Toolbar	New project, Open, Save all, New block, Tags, Compile, Build, Variables, All elements, Project tree, SCL elements, CPU
Project tree (Alt+1)	PLC tags, UDT, Program blocks (OB/FB/FC/DB พร้อมเลข), Source files; context menu: open, interface, set number, remove
SCL editor	syntax highlight, auto highlight ชื่อ, Find Declaration (คลิกขวา / Alt+Up / Ctrl+Click), Go Back (Alt+Left), Find, Undo/Redo
SCL elements (Alt+2)	20 หมวด พับ/ขยาย ค้นหา ลากวาง / ดับเบิลคลิก, หมวด Project อัปเดตตาม compile
Messages	error/warning ดับเบิลคลิกไปบรรทัด
Views	All variables, All elements, Listing, Tags, Interface grid
File	Recent projects / Recent files, import tags, add source

UI พื้นฐานที่ IDE ต้องเพิ่มเพื่อทำงานกับ Virtual PLC

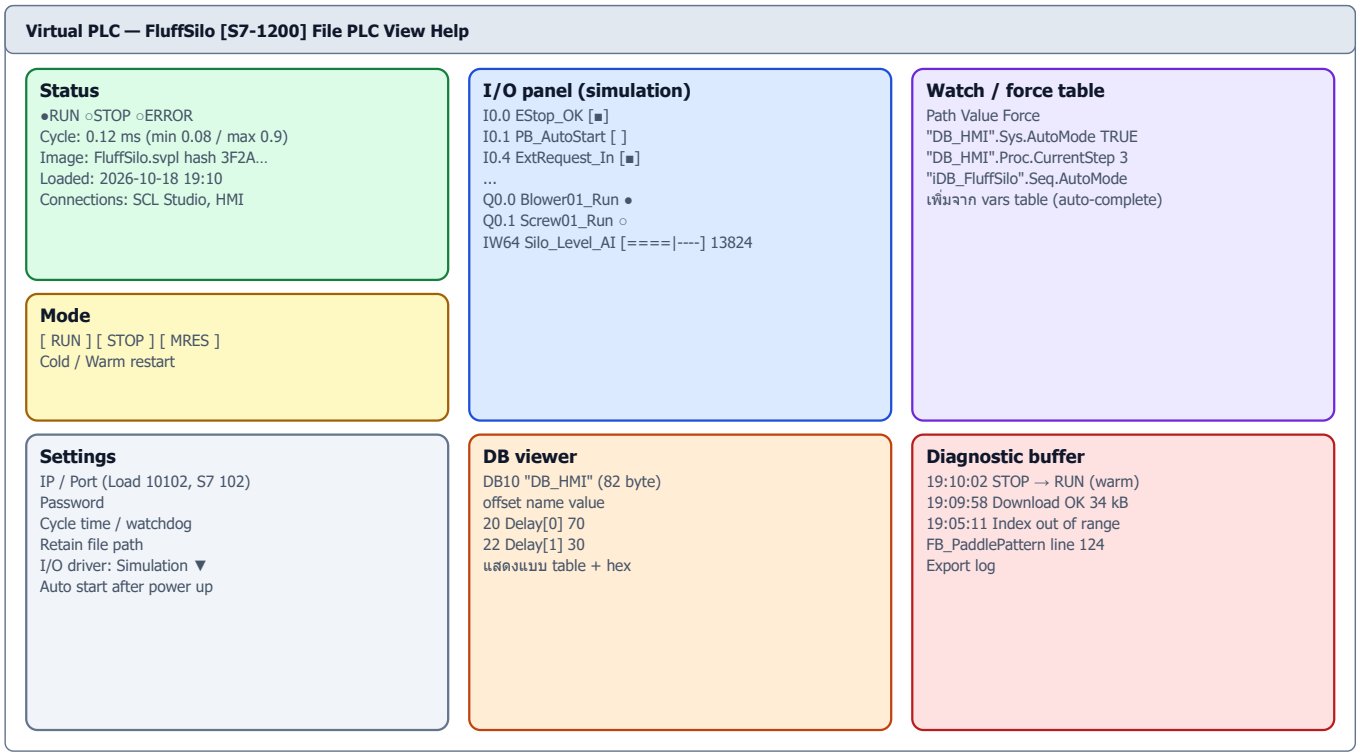
UI ที่ IDE ต้องเพิ่มสำหรับ Online / Download (wireframe)


File Edit PLC Build Online▼ View Help [Connect] [Download] [RUN] [STOP] [Monitor]
Online menu
Connect / Disconnect (IP, password)
Download (F10) · Upload
Compare offline / online (hash)
RUN / STOP / Cold restart
Diagnostic buffer
Editor online status
ค่า variable ข้างบรรทัด (line table)
สีเขียว/เทา ของเงื่อนไข IF
Hover = ค่า + type
Watch from cursor
Watch table tab
เพิ่ม path จาก Find Declaration
Monitor ทุก 200 ms
Modify / Force
บันทึก .watch
Download dialog
Target: 192.168.0.10 CPU S7-1200 ✓ (ตรงกับโปรเจกต์)
Blocks: 11 code, 2 DB ขนาด 34 kB hash 3F2A…
ตัวเลือก: [x] STOP ก่อนโหลด [x] เก็บค่า retain [ ] Reset DB start values [x] START หลังโหลด
Progress ▓▓▓

ลำดับความสำคัญ	UI	รายละเอียด
กลาง	Compare offline/online	hash ทั้งไฟล์และต่อ block
ต่ำ	Upload	ดึง image จาก Virtual PLC (ดูได้แต่ไม่มี source)

UI พื้นฐานของ Virtual PLC

UI พื้นฐานของ Virtual PLC (wireframe)



รูป 8.9 Virtual PLC UI (wireframe)

ส่วน	ต้องมี
Status	RUN / STOP / ERROR LED, cycle time min/avg/max, ชื่อ image, hash, เวลาที่โหลด, จำนวน connection
Mode	ปุ่ม RUN / STOP / MRES, cold / warm restart, auto start เมื่อเปิดโปรแกรม
I/O panel	checkbox ของ input (จาก TAGS), lamp ของ output, slider ของ analog — ใช้ทดสอบเมื่อไม่มี hardware
Watch / force	ตารางตัวแปรตาม path พร้อมค่า และ force
DB viewer	ดู DB เป็นตาราง (จาก vars) และ hex
Diagnostic buffer	ประวัติเหตุการณ์ export ได้
Settings	port, password, cycle/watchdog, retain path, I/O driver, S7 server on/off

แหล่งข้อมูลของหัวข้อนี้

- ภาพหน้าจอจากการรัน bin/SCLStudio.exe จริง
- wireframe ออกแบบใหม่ในเอกสารนี้
- แนวคิด UI ของ TIA Portal (Online & diagnostics, Watch table, Force table) และ PLCSIM เป็นแรงบันดาลใจ — ไม่ได้คัดลอกหน้าจอ

การทดสอบ

ชุด	ไฟล์	ตรวจอะไร	ผล
FluffSilo	examples/FluffSilo/tests/sequence.txt	checklist หน้างาน 12 ข้อ: E-Stop, Auto Start/Stop, delay, Pattern, fault, HMI command, analog, clamp	65/65
FeatureTest	examples/FeatureTest/tests/features.txt	CASE WHILE REPEAT FOR BY EXIT CONTINUE String UDT array 2D array FC output/InOut CTU TP IEC_TIMER MUX SHL ROL slice Time overflow	44/44
ErrorTest	examples/ErrorTest/errors.scl	narrowing, range, unknown, index, condition type, Time, unknown FC, missing ;, EXIT, SHL real	10/10
CPU rules	FeatureTest --cpu S7-300 / S7-1200	LReal/LTime/slice ถูกปฏิเสธตาม CPU	ผ่าน
Round trip	grid → source ทุก block แล้ว compile + test	declgen ไม่ทำให้ความหมายเปลี่ยน	65/65
GUI	ภาพหน้าจอจากการทดสอบด้วยเมาส์/คีย์บอร์ด	tree, editor, views, find decl, drag & drop, recent	ผ่าน (manual)

ภาษา script (sclc --sim image.svpl --script file.txt)

คำสั่ง	ความหมาย
cycle-ms 10	เวลาจำลองต่อ cycle
set <path> <value>	เขียน tag หรือ "DB".path
link <input> <output>	input = output ทุก cycle (จำลอง feedback)
run 7s | run 500ms | run 3 cycles	รัน
expect <path> <value>	PASS / FAIL
print <path>	แสดงค่า
echo text	ข้อความ

วิธีเพิ่ม test

1. สร้างโปรเจกต์ใน examples/<ชื่อ> (หรือใช้ sclc --import)
2. เขียน tests/<ชื่อ>.txt ด้วยคำสั่งข้างบน
3. เพิ่มใน run_tests.bat
4. ทุกครั้งที่แก้ codegen หรือ opcode ให้รัน run_tests.bat

> แนะนำเพิ่ม: เทียบผลกับ PLCSIM / PLC จริงด้วย script เดียวกัน (golden test) และ CI (GitHub Actions + Lazarus) เพื่อรันอัตโนมัติ

แหล่งข้อมูลของหัวข้อนี้

- ไฟล์ทดสอบในโฟลเดอร์ examples
- README ของ PLC_S7-1200 (checklist หน้างาน)

คู่มือพัฒนาต่อ

Build

```
build.bat                       :: CLI + GUI
fpc -Mobjfpc -Sh -gl -Fucore -FUlib\cli -obin\sclc.exe cli\sclc.lpr
lazbuild gui\SCLStudio.lpi       :: GUI (package LCL + SynEdit)
```

เพิ่ม builtin function ใหม่

1. เพิ่มชื่อใน IsBuiltinName (scl_compiler)
2. กำหนดชนิด argument/ผลลัพธ์ใน InferBuiltin
3. สร้าง bytecode ใน EmitBuiltin (ใช้ opcode เดิม หรือเพิ่ม BI_xxx ใน opBuiltin)
4. ถ้าเพิ่ม BI_xxx: implement ใน scl_vm.pas และบันทึกใน SVPL_Spec
5. เพิ่ม element ใน sclide_elements (BuildStatic) และ test ใน FeatureTest

เพิ่ม opcode

1. เพิ่มท้าย TOpcode (อย่าแทรกกลาง — เลข opcode เปลี่ยน)
2. กำหนด operand ใน OpOperands
3. implement ใน TVirtualPlc.Execute
4. ปรับ SVPL_VERSION ถ้าไม่เข้ากันย้อนหลัง
5. อัปเดต docs/SVPL_Spec.md และหัวข้อ 06

เพิ่มชนิดข้อมูล / system FB

- ชนิด: TTypeKind (เลขต่อจากเดิม = VM type code), ElementaryBits, TypeSupportedOn, Load/Store ใน scl_vmmem
- System FB: ตาราง SysFBs + MakeSysFBType (layout), SysFBId, implement ใน TVirtualPlc.SysFB, เพิ่ม element ใน panel

เพิ่ม panel / แท็บใน GUI

- แท็บ editor ใหม่: สืบทอด TEditorTab (sclide_editors) ตั้ง Kind/Key/BaseCaption
- GUI สร้างด้วยโค้ด (ไม่มี .lfm) — form ใช้ CreateNew
- ผูกกับ compile: เรียกใน RunCompile หลัง RebuildTree

Coding convention

- {$mode objfpc}{$H+}, ชื่อ field ขึ้นต้น F, type ขึ้นต้น T, ห้ามใช้ with ซ้อน
- core ห้าม uses LCL
- ข้อความ error เป็นภาษาอังกฤษสั้น มีชื่อ/ค่า
- ทุก unit core มี header comment บรรทัดแรก

ข้อควรระวัง (เจอจริงระหว่างพัฒนา)

ปัญหา	วิธีแก้
ใน function ที่ไม่มี parameter ชื่อ function = ตัวแปร Result (ParseUnary; ไม่ recurse)	เรียกตัวเองด้วย ParseUnary()

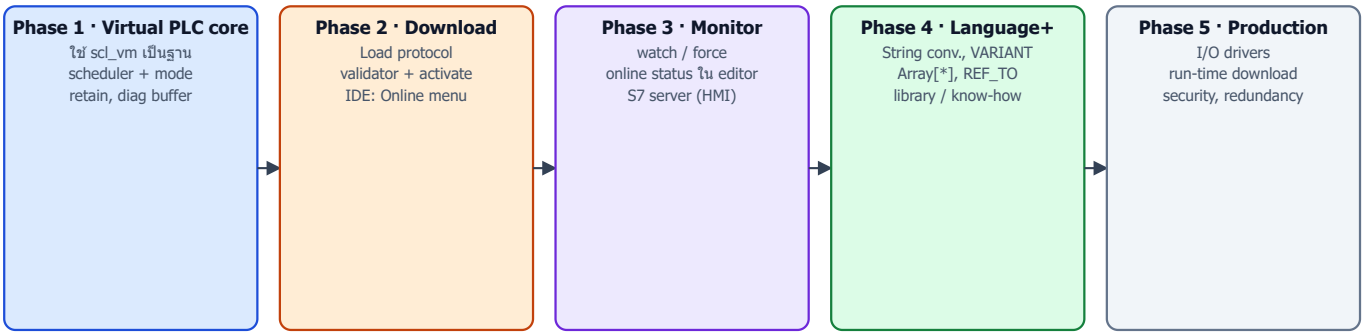
ปัญหา	วิธีแก้
ชื่อ local/parameter ชนกับ property ของ LCL (Text, Left, Top, Key)	ใช้ AText, PLeft, OldTop, AKey
SynEdit ใช้ Ctrl+0..9 (bookmark) และกิน Alt+key	shortcut ระดับ app ผ่าน Application.AddOnKeyDownBeforeHandler
SynEdit word break ไม่มี # → #Name เป็นคำเดียว	เพิ่ม '#', '%', '"' ใน WordBreakChars ของ highlighter
สร้าง form โดยไม่มี .lfm	constructor Create override → inherited CreateNew
for..in กับ array literal ใช้ไม่ได้ใน FPC 3.2	ใส่ใน array variable ก่อน
ทดสอบ GUI ด้วย SendKeys: Alt+1 ส่งเป็น VK_NUMPAD1	รองรับทั้ง VK_1 และ VK_NUMPAD1

แหล่งข้อมูลของหัวข้อนี้

- Free Pascal Reference Guide และ Lazarus wiki (SynEdit, LCL, lazbuild)
- ประสบการณ์แก้ปัญหาระหว่างพัฒนาโปรเจกต์นี้

คำแนะนำและ Roadmap

Roadmap แนะนำ



ลำดับแนะนำ: ทำ Phase 1+2 ให้ใช้งานได้ก่อน แล้วค่อยเพิ่ม monitor และ feature ภาษา

รูป 11.1 Roadmap แนะนำ

คำแนะนำสำคัญ

1. ตรวจกับของจริงก่อนขยาย: import FluffSilo เข้า TIA Portal เทียบ offset ของ DB และพฤติกรรม (PLCSIM) เพื่อยืนยันกฎ layout/conversion
2. สร้าง Virtual PLC จาก scl_vm.pas ก่อน (ทำงานถูกแล้วกับ 109 checks) แทนการเขียนใหม่
3. ทำ Download แบบ STOP-only ก่อน, run-time download ไว้ทีหลัง
4. เพิ่ม layout hash ต่อ DB และ hash ทั้ง image (SVPL v2) เพื่อ keep retain และ compare online/offline อย่างปลอดภัย
5. ล็อก format: ทุกการเปลี่ยน opcode/section ต้องเพิ่ม version และมี test
6. ความปลอดภัย: Virtual PLC ห้ามใช้แทน safety PLC / E-Stop ต้องเป็น hardware; เปิด password และจำกัด network ของ PUT/GET
7. ประสิทธิภาพ: ถ้า cycle ยาว ให้ทำ pre-decode bytecode เป็น array ของ record (ไม่ต้องอ่าน operand ทุกครั้ง) ก่อนคิด JIT
8. License: Lazarus LCL (modified LGPL), SynEdit (MPL/GPL dual), Snap7 (LGPL3) — ตรวจเงื่อนไขก่อนแจกจ่ายเชิงพาณิชย์

ความเสี่ยง

ความเสี่ยง	ผลกระทบ	การลด
พฤติกรรมต่างจาก S7 จริง (rounding, overflow, timer edge)	โปรแกรมทำงานต่างกัน	golden test กับ PLCSIM
Feature ภาษาที่ยังไม่รองรับ	import โปรแกรมเดิมไม่ได้	จัดลำดับตามโปรแกรมจริงที่ใช้
Download ทำ image เสีย	เครื่องหยุด	validator + atomic swap + เก็บ image เดิม
Retain หาย	ค่าตั้งหาย	retain.bin + CRC + layout hash
Network ไม่ปลอดภัย	มีคนแก้โปรแกรม	password, session lock, จำกัด IP

Roadmap

Phase	งาน	ผลลัพธ์
1	Virtual PLC core: host, mode, scheduler, retain, diag, UI พื้นฐาน, I/O simulation	รัน .svpl ได้เอง

Phase	งาน	ผลลัพธ์
2	Load protocol + validator + IDE Online menu + Download dialog	กด Download จาก SCL Studio ได้
3	Watch/force, online status ใน editor, S7 server (Snap7) ให้ HMI	debug online, ใช้ HMI เดิม
4	String conversion, VARIANT/Array[*], library, know-how protection	รองรับโปรแกรมได้กว้างขึ้น
5	I/O driver จริง, run-time download, redundancy, security	ใช้งานจริง

แหล่งข้อมูลของหัวข้อนี้

- ผลการทดสอบในหัวข้อ 09
- ข้อจำกัดในหัวข้อ 03
- license ของ Lazarus / SynEdit / Snap7 จากเว็บไซต์ของแต่ละโครงการ (ควรตรวจฉบับล่าสุด)

แหล่งข้อมูลที่ใช้สร้าง

> ความโปร่งใส: โปรแกรมและเอกสารนี้สร้างจากความรู้ของผู้เขียน (AI) ร่วมกับโค้ดและไฟล์ในเครื่องของผู้ใช้ ระหว่างพัฒนาไม่ได้ดาวน์โหลดเอกสารจากอินเทอร์เน็ต รายการเอกสารภายนอกด้านล่างคือแหล่งความรู้มาตรฐานของแต่ละเรื่อง และเป็นเอกสารที่ควรใช้ตรวจสอบ — กรุณาคันฉบับล่าสุดจากผู้ผลิต

ข้อมูลภายในโปรเจกต์ (ใช้จริง)

แหล่ง	ใช้สร้างหัวข้อ
PLC_S7-1200/ (14 ไฟล์ SCL, tag CSV, README)	ตัวอย่างจริง, test FluffSilo, layout DB_HMI, checklist
PLC_S7-1200_Analysis/ (รายงานวิเคราะห์)	ความเข้าใจระบบ FluffSilo และข้อสังเกต
SCLCompiler/core, gui, cli	ทุกหัวข้อที่อธิบายโค้ด
ผลรัน run_tests.bat, ภาพหน้าจอ SCLStudio.exe	หัวข้อ 01, 08, 09
Lazarus source (C:\lazarus\components\synedit)	การใช้ SynEdit: highlighter, markup highlight-all, drag & drop

มาตรฐานและเอกสารผู้ผลิต (ควรใช้ตรวจสอบ)

เรื่อง	เอกสาร
ภาษา ST/SCL	IEC 61131-3:2013 Programmable controllers — Part 3: Programming languages; PLCopen (plcopen.org)
SCL ของ S7-300/400	Siemens "SIMATIC S7-SCL for S7-300/S7-400" Manual
SCL / data types / instructions ของ S7-1200/1500	Siemens TIA Portal STEP 7 Information System (Help) และ "S7-1200 Programmable controller System Manual", "S7-1500/ET 200MP System Manual"
แนวทางเขียนโปรแกรม	Siemens "Programming Guideline for S7-1200/S7-1500" และ "Programming Styleguide for S7-1200/S7-1500"
External source format	TIA Portal Help: "Generating blocks from source" / "SCL external source files"
Memory areas / standard vs optimized access	TIA Portal Help: "Optimized block access", "Data storage"
OB / operating modes	System manual ของ S7-1200 และ S7-1500 หัวข้อ Organization blocks และ Operating modes
เว็บไซต์ค้นเอกสาร Siemens	support.industry.siemens.com (Siemens Industry Online Support)

Protocol / communication

เรื่อง	แหล่ง
ISO-on-TCP	RFC 1006 (rfc-editor.org), ISO 8073 / COTP
S7 protocol, PUT/GET	Snap7 (snap7.sourceforge.net) documentation; Wireshark S7comm dissector
Socket ใน Free Pascal	FPC FCL fcl-net (ssockets) documentation; Synapse library (synapse.ararat.cz)
Checksum / hash	CRC-32 (IEEE 802.3), SHA-256 (NIST FIPS 180-4)

Compiler / VM / tools

เรื่อง	แหล่ง
ทฤษฎี compiler	Aho, Lam, Sethi, Ullman — Compilers: Principles, Techniques, and Tools

เรื่อง	แหล่ง
Stack VM / interpreter	Robert Nystrom — Crafting Interpreters (craftinginterpreters.com)
Free Pascal	freepascal.org/docs.html (Reference guide, RTL, FCL)
Lazarus / LCL / SynEdit	wiki.lazarus.freepascal.org (SynEdit, LCL, lazbuild)
IEEE-754	IEEE Std 754 (Real / LReal)

แหล่งข้อมูลต่อหัวข้อ

แต่ละไฟล์หัวข้อมีส่วน "แหล่งข้อมูล" ท้ายหัวข้อ ระบุแหล่งที่ใช้สร้างหัวข้อนั้น

ภาคผนวก

Glossary

คำ	ความหมาย
SCL	Structured Control Language — ภาษาแบบ ST (IEC 61131-3) ของ Siemens
OB / FB / FC / DB / UDT	Organization block / Function block (มี instance) / Function / Data block / PLC data type
Instance DB	DB ที่เก็บหน่วยความจำของ FB หนึ่งตัว
Multi-instance	FB ซ้อนอยู่ใน static ของ FB อื่น
Standard / optimized access	DB ที่มี offset ตายตัว (HMI ใช้ address ได้) / ไม่มี offset ภายนอก
Retain	ค่าที่คงอยู่หลังไฟดับ
Image (.svpl)	ไฟล์ผลลัพธ์ของ compiler ที่ Virtual PLC โหลด
Pending frame	frame ของ FC ที่ caller จองเพื่อใส่ parameter ก่อนเรียก
Download	ส่ง image จาก IDE ไปยัง Virtual PLC
PUT/GET	บริการอ่าน/เขียนหน่วยความจำของ S7 ที่ HMI ใช้

รูปแบบไฟล์ .sclproj

```
[Project]
Name=FluffSilo
CPU=S7-1200
OutputDir=output
[Sources]
File1=02_UDT_typeDevStatus.scl
[Tags]
File1=01_PLC_Tags.csv
[BlockNumbers]
DB_HMI=10
[CyclicInterval]
Cyclic interrupt=100
```

รูปแบบ tag CSV

```
Name;Path;Data Type;Logical Address;Comment;Value
EStop_OK;FluffSilo_IO;Bool;%I0.0;Safety relay OK;
MAX_SPEED;Constants;Int;;user constant;1500
```

คำสั่ง CLI

คำสั่ง	ผล
sclc p.sclproj [-o x.svpl] [-l x.lst] [--check] [--cpu S7-1500]	compile
sclc --import <folder> --cpu S7-1200 -p p.sclproj	สร้างโปรเจกต์จากโฟลเดอร์
sclc --dump x.svpl	แสดงเนื้อหา image
sclc --sim x.svpl --script t.txt	รันทดสอบ
sclc p.sclproj --check --find-decl file:line:col	ทดสอบ Find Declaration

คีย์ลัด SCL Studio

คีย์	คำสั่ง
Ctrl+O / Ctrl+S	Open project / Save all
Ctrl+N	New block
Ctrl+T	Tags
F7 / F9	Compile / Build
F11 / F12	All variables / All elements
Alt+1 / Alt+2	Project tree / SCL elements panel
Alt+Up, Ctrl+Click	Find Declaration
Alt+Left	Go Back
Ctrl+F / F3	Find / Find next
Ctrl+W	Close tab
Enter / Ctrl+Enter (tree)	เปิด source / interface editor

แหล่งข้อมูลของหัวข้อนี้

- โค้ด scl_project.pas, sclc.lpr, sclide_main.pas